

Integrated Gene Knowledge Database

Course Section: EN.605.641.81
Spring, 2026

Prepared by

Anish Raja
4/11/2026

Table of Contents

1.	INTRODUCTION.....	3
1.1.	SCOPE AND PURPOSE OF DOCUMENT.....	3
1.2.	PROJECT OBJECTIVE.....	3
2.	SYSTEM REQUIREMENTS.....	3
2.1	HARDWARE REQUIREMENTS	3
2.2	SOFTWARE REQUIREMENTS.....	4
2.3	FUNCTIONAL REQUIREMENTS	4
2.4	DATABASE REQUIREMENTS.....	4
3.	DATABASE DESIGN DESCRIPTION	4
3.1	DESIGN RATIONALE	4
3.2	ER MODEL	6
3.2.1	<i>Entities</i>	<i>6</i>
3.2.2	<i>Relationships.....</i>	<i>8</i>
3.2.3	<i>ER Diagram.....</i>	<i>11</i>
3.3	RELATIONAL MODEL	12
3.3.1	<i>Data Dictionary.....</i>	<i>12</i>
3.3.2	<i>Integrity Rules.....</i>	<i>17</i>
3.3.3	<i>Operational Rules</i>	<i>19</i>
3.3.4	<i>Operations</i>	<i>19</i>
3.4	SECURITY	20
3.5	DATABASE BACKUP AND RECOVERY	22
3.6	USING DATABASE DESIGN OR CASE TOOL	23
3.7	OTHER POSSIBLE ER RELATIONSHIPS	23
4.	IMPLEMENTATION DESCRIPTION	24
4.1	DATA DICTIONARY	24
4.2	ADVANCED FEATURES	24
4.3	QUERIES	49
4.3.1	<i>Gene Products Associated with a Genetic Disease.....</i>	<i>49</i>
4.3.2	<i>Total Genes in a Gene Family for Each Organism.....</i>	<i>50</i>
4.3.3	<i>Finding Pathways Associated with a Genetic Disease</i>	<i>51</i>
4.3.4	<i>Gene Annotations Meeting Gene Product and GO Term Criteria.....</i>	<i>52</i>
4.3.5	<i>Gene Product and GO Term Annotation Status.....</i>	<i>53</i>
4.3.6	<i>Retrieve Child Gene Ontology Terms</i>	<i>54</i>
4.3.7	<i>Retrieve Gene Annotations with Recent Sources</i>	<i>55</i>
4.3.8	<i>Gene Products for Genes Found in Region of Chromosome</i>	<i>56</i>
5.	CRUD MATRIX	57
5.1	LIST OF ENTITY TYPES	57
5.2	LIST OF FUNCTIONS.....	57
6.	CONCLUDING REMARKS.....	59
	APPENDICES.....	60
	REFERENCES.....	60

1. Introduction

In the field of bioinformatics, databases are fundamental for storing, managing, and retrieving vast amounts of biological data. Public databases, such as GenBank and Ensembl, handle genomic data while providing important functionality such as annotations, standardization, and cross-referencing. The inspiration for this database project is one such database, Chado. Chado is a community-driven, open-source relational database schema that is highly normalized and can be scaled to accommodate many different types of biological data. I have previously worked with Chado in a bioinformatics course to analyze prokaryotic gene expression data and annotations and developed a gene search program that interacts with this database. What makes Chado so appealing is its modular nature, which allows users to select only the components they need. These modules support various types of biological data, including sequence information, taxonomic data, controlled vocabularies (CVs), and ontologies. This project focuses on gene annotations and their association with ontology terms in particular. Controlled vocabularies provide a standardized and consistent way to describe data, while ontologies define structured relationships between these terms. Gene annotations are an important component of biological databases as they provide meaningful interpretation of molecular function, location, and biological significance for genes and their products. Since genes are clearly defined, hierarchical, and share conserved functionality across species, they serve as an ideal foundation for a structured database. The database of this project, the Integrated Gene Knowledge Database (IGKD), is designed to store, organize, and manage gene annotation data, linking gene products to gene ontology terms. Additionally, the database incorporates temporal information to track when gene annotations are created and updated, supporting data accuracy and reproducibility in biological research.

1.1. Scope and Purpose of Document

In this document, we will present the requirements, design, and implementation of the Integrated Gene Knowledge Database. We begin with an overview of system requirements, including hardware, software, functional, and database requirements. The conceptual design is then discussed through the design rationale and Entity Relationship (ER) model, including entities, relationships, and the ER diagram. The document will also describe the logical design using the relational model, including the data dictionary, integrity rules, operational rules, and database operations. Additionally, topics such as database security, backup and recovery, and the CASE tools that were used in development are addressed. Advanced features and queries implemented in the system are also presented. Finally, we will conclude with the CRUD matrix and some final remarks.

1.2. Project Objective

The objective of this database project is to design and implement the Integrated Gene Knowledge Database using a relational DBMS. The database captures and manages key aspects of gene-related information and has a primary focus on gene annotations and how they establish relationships between gene products and ontology terms. There is support for operations that store, retrieve, and organize biological data, enabling efficient querying and analysis of gene functions, relationships, and their associated metadata.

2. System Requirements

The general system requirements covering hardware, software, functional and database requirements are listed below. These requirements define the operation environment and capabilities necessary for the system to function effectively.

2.1 Hardware Requirements

The minimum hardware requirements are:

- **CPU:** Multi-core processor (e.g. Intel i5 or equivalent) for handling database operations.
- **Memory:** 8 GB RAM for query caching and performance.
- **Storage (Hard Disk / SSD):** 5-10 GB of storage for storing database tables and indexing. SSD is preferred.

2.2 Software Requirements

The minimum software requirements to run this application are:

- **OS Requirements:** Windows 10 or later, macOS 11 or later
- **MySQL Server and Workbench:** Version 8.0 or later

2.3 Functional Requirements

The database supports a variety of functional requirements which are shown below:

1. Users can look up gene products that are associated with a particular genetic disease. When a disease is specified, the genes associated with that disease are identified along with their corresponding gene products. This allows users to understand how certain genes and their products are involved in the disease process.
2. Users can determine how many genes from a particular gene family are present in each organism. This allows for gene family distributions to be compared across different species, which can help identify conservation/variation patterns among organisms.
3. Users can identify the biological pathways associated with genes that are linked to a specific disease. When a disease is chosen, pathways involving the disease-related genes are returned. This allows users to gain a better understanding of the mechanisms and interactions that underlie the disease.
4. Users can retrieve gene annotations by providing criteria such as the organism, chromosome, and ontology category. This allows users to focus their analysis on gene function, biological processes, or cellular components within a certain genomic region and species.
5. Users can evaluate whether certain combinations of gene products and gene ontology term pairs are annotated or missing an annotation for a given organism. This allows users to identify incomplete annotation data and also highlights areas where further research may be needed.
6. Users can retrieve all child gene ontology terms associated with a particular parent term. This provides users with a hierarchical view of relationships.

2.4 Database Requirements

The database used for this project is MySQL Version 8.0.45.

3. Database Design Description

In this section, we will rationalize the database design, cover the ER Model in detail, and discuss the relational model. We will also explain how security was implemented, address database backup and recovery, and describe usage of any tools in development.

3.1 Design Rationale

The database stores information about genes, so the gene will be the central entity. Some basic information stored about each gene includes the gene symbol, a short description, the name of the chromosome in which it is found, the starting base-pair position, the ending base-pair position, and the strand it is a part of. Genes can be found in both the forward and reverse strand, so this needs to be specified. A gene symbol is an abbreviation for a gene name that is unique within a species but may be shared across species. For this reason, and because gene symbols can change over time, it is not used as the primary key. Instead, the surrogate key, `gene_id`, will be used as the primary key. A surrogate key is used as the primary key because it only needs to act as a unique identifier and doesn't need to hold any real-world meaning. This is true for all surrogate keys used in this database.

Since we need some way of knowing which species a gene is from, we will have an organism entity. It will store information such as the scientific name of the species, the size of the genome, the total number of chromosomes, the domain, and the taxonomy id. While the scientific name is unique for a species, it can change over time, so it is not used as the primary key. The taxonomy id isn't used either as this database isn't limited to genes from organisms listed in the NCBI database. The main purpose of the taxonomy id is to act as a stable, standardized identifier for organisms when it is available. It is a widely adopted standard

in the bioinformatics field, so it will serve a lot of functionality (i.e. with lookups) in this database. A surrogate key, `organism_id`, is used as the primary key. A given gene can only be associated with one organism, but an organism can contain 0 or more genes. This is to allow for an organism entity to be created before genes are added to it.

The expression of a gene often leads to the formation of a gene product, so a `gene_product` entity is created. It will store information such as the name of the gene product, the product type, and a description. Since multiple genes can produce the same product, `product_name` does not have to be unique. Thus, a surrogate key, `gene_product_id`, is used as the primary key. A gene can produce 0 or more gene products, but a gene product must be associated with exactly one gene. The reason why the same gene product can exist multiple times is because the gene product associated with one gene may exhibit different functionality compared to another gene. This is especially useful for gene annotations where the gene product needs to be referenced with respect to the gene it is from. This is why the `GENE_ANNOTATION` entity references the `gene_product_id` attribute and not the `product_name` attribute.

Gene ontology terms, or GO terms, provide a standardized description of gene function and are essential for representing functional annotations in a way that is structured and consistent, thus a `GO_TERM` entity is created. There are 7 mandatory elements pertaining to go terms that need to be included: a unique identifier, the name of the term, the ontology the term falls under, a definition of the term, the date the go term was created, the date the go term was last updated, and any relationships with other terms. The unique identifier, `go_id`, will serve as the natural primary key. This key must adhere to official naming conventions set by the Gene Ontology Consortium, which is the format 'GO:0000000', where '0' is used as a placeholder for any digit. The term name must be readable by a human and be self-explanatory. A definition is provided for a more detailed description. A GO term can fall under 3 possible sub-ontologies: Molecular Function (MF), Biological Process (BP), and Cellular Component (CC). The date of creation and last update are stored to track changes and ensure data accuracy over time. To store information on the relationship between two go terms, we need to create a recursive relationship and a separate intersecting entity, `GO_TERM_RELATION`. This entity will contain the 2 GO ids of the GO terms with a relationship. A relation term attribute will describe the relationship between the 2 GO terms and can be one of five values: "is a", "part of", "has part", "regulates", and "occurs in". While "is a" can be used to model the inheritance found in superclass/subclass relationships within EERDs, the other semantics cannot, so this intersection entity is necessary. It is important to note that each GO term can have multiple "parent" and "child" terms. GO terms follow a structural hierarchy that can best be represented as a directed acyclic graph. The "parent" of a GO term is one that connects to it and represents a higher-level concept. Similarly, a "child" of a GO term is one that connects to it and represents a lower-level concept. This structure we have defined here will allow users to see what the "parent" and "child" terms of any particular GO term are. A GO term relation is only between 2 GO terms, one being the parent and the other the child, but a GO term can participate in multiple GO term relations.

The fundamental entity of this database, `GENE_ANNOTATION`, represents a gene "annotation" and is a child entity of its two parent entities, `GENE_PRODUCT` and `GO_TERM`. This associative entity functions as an "annotation" in the following sense: it tells the user that "gene X has function Y" where "X" is associated with a gene product and "Y" is associated with a GO term. In other words, a gene annotation connects a gene product with a GO term. This entity also contains information on the evidence code, date of creation, date of last update, and any comments. Since this entity captures the association between its two parent entities, holds relationship-specific attributes, and resolves the M:N relationship between `GENE_PRODUCT` and `GO_TERM` (as a gene product can be annotated by zero or many GO terms and a GO term can annotate zero or more genes products), it is necessary as an intersection entity. The evidence code indicates how the annotation to a GO term is supported. There are 25 different evidence codes defined by the Gene Ontology Consortium. The `date_created` and `date_last_update` attribute details the exact date and time the user created and last updated the annotation. The `comment` attribute allows a researcher to describe the relationship between the gene product and GO term in more detail. It is also important for a gene annotation to contain a source. A source attribute wouldn't be sufficient as a gene annotation can contain multiple sources, meaning it would have to be a multivalued attribute, and this would violate the first normal form. Instead, a separate `SOURCE` entity is created with an intersecting entity called `ANNOTATION_SOURCE` that resolves the M:N relationship between the gene annotation and source. A

gene annotation must be associated with at least 1 source, and a source must be attributed to at least 1 gene annotation. The annotation source entity has a primary key composed of the foreign keys `source_id` and `annotation_id`. The primary key of the gene annotation entity, `annotation_id`, is a surrogate key. A gene annotation must contain a unique pairing of gene product and GO term, and this entity does not include `gene_product_id` or `go_id` in its primary key. Instead, a uniqueness constraint will be applied on both of these foreign key attributes to ensure that only one annotation is made per combination of gene product associated with a particular gene and GO term.

The source of a gene annotation can be a publication, an external database, or any other source. As long as the source has a name, url path, and a date of publishing, it is a valid source. This IGKD database requires that every source contains a url to maintain the robustness of the data; a user should be able to visit the url and both confirm the validity of the source and visit the source to understand for themselves why it serves as evidence for the gene annotation. Knowing the year of publication for a source allows users to determine how relevant the source material is. The primary key of the SOURCE entity is the surrogate key, `source_id`.

This database also stores information related to gene families, pathways, and diseases. Thus, the GENE_FAMILY, GENE_PATHWAY, and GENE_DISEASE entities are created. A gene family is essentially a set of homologous genes; they share ancestry and members of the family often contain structural and functional similarities. The entity also stores information on the name of the family and its description. The primary key of this entity, `family_id`, is a surrogate key. While the name of the gene family must be unique, it can change over time, hence why it is not used as the primary key. Since a gene may be a part of one or more gene families, and a gene family must contain at least one gene, an intersection entity called GENE_FAMILY_MEMBER is needed to resolve this M:N relationship. This intersection entity has a composite primary key composed of the foreign keys `gene_id` and `family_id`. A gene pathway contains a group of genes that interact with each other to carry out a specific biological process or function. This entity contains information on the name of the pathway and a description. Similar to a gene family, the name of the pathway must be unique, and can change over time, so it is not used as the primary key. The primary key of this entity, `pathway_id`, is a surrogate key. Since a gene may be a part of one or more pathways, and a gene pathway must contain at least one gene, an intersection entity called GENE_PATHWAY_ASSOCIATION is needed to resolve this M:N relationship. This entity has a composite primary key composed of the foreign keys `gene_id` and `pathway_id`. A gene disease is a disease that is associated with a gene. This entity contains information on the name of the disease and category the disease belongs to. Similar to the gene family and gene pathway entities, the name of the disease must be unique and can change over time, so it is not used as the primary key. The primary key of this entity, `disease_id`, is a surrogate key. Since a gene may be associated with one or more diseases, and a disease must be associated with at least one gene, an intersection entity called GENE_DISEASE_ASSOCIATION is needed to resolve this M:N relationship. This entity has a composite primary key composed of the foreign keys `gene_id` and `disease_id`. This database will only keep track of diseases, gene families, and gene pathways that are related to genes that exist in this database, hence the total participation.

3.2 ER Model

In this section, we will discuss entities and their relationships in more detail and present the ER Diagram. The structure of the database will be illustrated to show how the entities are connected with each other and how the data is organized.

3.2.1 Entities

Now, we will cover each of the entities in more detail. The design rationale for including each entity and what they represent have been explained in section 3.1.

1. GENE
 - Strong Entity
 - Primary Key Attributes:
 - `gene_id` (surrogate key)
 - Non-key Attributes:
 - `organism_id` (foreign key, reference to ORGANISM entity)

- gene_symbol
 - short_description
 - chromosome
 - start_position
 - end_position
 - strand
- 2. ORGANISM
 - Strong Entity
 - Primary Key Attributes:
 - organism_id (surrogate key)
 - Non-key Attributes:
 - scientific_name
 - genome_size
 - chromosome_count
 - domain
 - taxonomy_id
- 3. GENE_PRODUCT
 - Strong Entity
 - Primary Key Attributes:
 - gene_product_id (surrogate key)
 - Non-key Attributes:
 - gene_id (foreign key, reference to GENE entity)
 - product_name
 - product_type
 - description
- 4. GENE_FAMILY
 - Strong Entity
 - Primary Key Attributes:
 - family_id (surrogate key)
 - Non-key Attributes:
 - family_name
 - description
- 5. GENE_FAMILY_MEMBER
 - Weak Entity
 - Primary Key (Composite Key) Attributes:
 - gene_id (foreign key, reference to GENE entity)
 - family_id (foreign key, reference to GENE_FAMILY entity)
- 6. GENE_DISEASE
 - Strong Entity
 - Primary Key Attributes:
 - disease_id (surrogate key)
 - Non-key Attributes:
 - disease_name
 - category
- 7. GENE_DISEASE_ASSOCIATION
 - Weak Entity
 - Primary Key (Composite Key) Attributes:
 - gene_id (foreign key, reference to GENE entity)
 - disease_id (foreign key, reference to DISEASE entity)
- 8. GENE_PATHWAY
 - Strong Entity
 - Primary Key Attributes:
 - pathway_id (surrogate key)
 - Non-key Attributes:
 - pathway_name
 - description

9. GENE_PATHWAY_ASSOCIATION
 - Weak Entity
 - Primary Key (Composite Key) Attributes:
 - gene_id (foreign key, reference to GENE entity)
 - pathway_id (foreign key, reference to GENE_PATHWAY entity)
10. GO_TERM
 - Strong Entity
 - Primary Key Attributes:
 - go_id (natural key)
 - Non-key Attributes:
 - term_name
 - ontology
 - definition
 - date_created
 - date_last_update
11. GO_TERM_RELATION
 - Strong Entity
 - Primary Key Attributes:
 - go_relation_id (surrogate key)
 - Non-key Attributes:
 - parent_go_id (foreign key, reference to GO_TERM entity)
 - child_go_id (foreign key, reference to GO_TERM entity)
 - relation_term
12. GENE_ANNOTATION
 - Strong Entity
 - Primary Key Attributes:
 - annotation_id (surrogate key)
 - Non-key Attributes:
 - gene_product_id (foreign key, reference to GENE_PRODUCT entity)
 - go_id (foreign key, reference to GO_TERM entity)
 - evidence_code
 - date_created
 - date_last_update
 - comment
13. SOURCE
 - Strong Entity
 - Primary Key Attributes:
 - source_id (surrogate key)
 - Non-key Attributes:
 - source_name
 - publish_year
 - url
14. ANNOTATION_SOURCE
 - Weak Entity
 - Primary Key (Composite Key) Attributes:
 - source_id (foreign key, reference to SOURCE entity)
 - annotation_id (foreign key, reference to GENE_ANNOTATION entity)

3.2.2 Relationships

Now, we will cover the relationships in detail. The design rationale for each relationship has been explained in section 3.1.

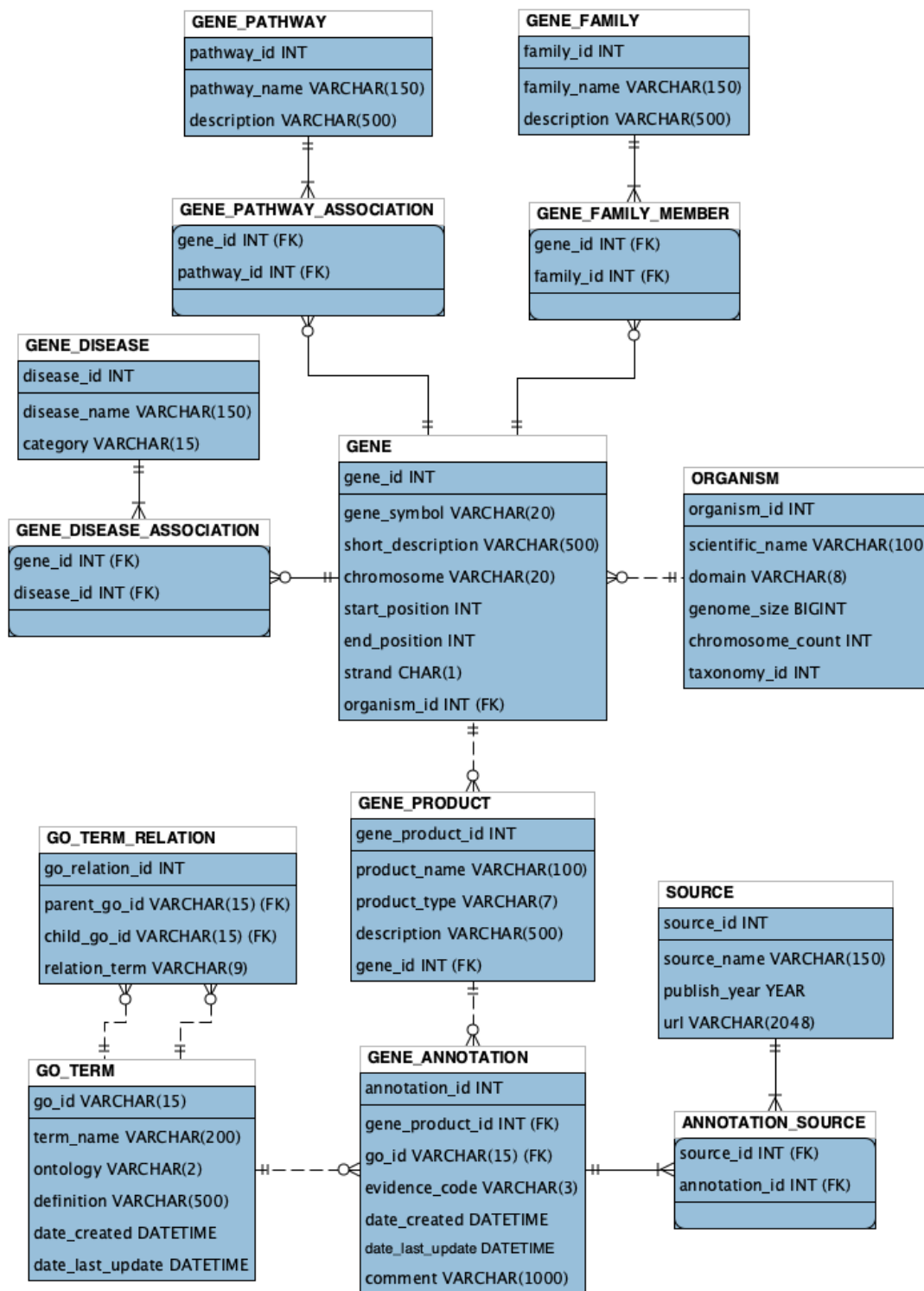
1. There is a 1:N relationship from ORGANISM to GENE.
 - Each organism may contain zero or more genes (partial participation).
 - Each gene must belong to exactly one organism (total participation).

- This relationship is non-identifying as both of these entities are strong entities and can exist without depending on the other type.
- 2. There is a 1:N relationship from GENE to GENE_PRODUCT.
 - Each gene may produce zero or more gene products (partial participation).
 - Each gene product must be associated with exactly one gene (total participation).
 - This relationship is non-identifying.
- 3. There is a 1:N relationship from GENE_PRODUCT to GENE_ANNOTATION.
 - Each gene product may contain zero or more annotations (partial participation).
 - Each gene annotation must correspond to exactly one gene product (total participation).
 - This relationship is non-identifying.
- 4. There is a 1:N relationship from GO_TERM to GENE_ANNOTATION.
 - Each gene ontology term may be included in zero or more gene annotations (partial participation).
 - Each gene annotation must correspond to exactly one gene ontology term (total participation).
 - This relationship is non-identifying.
- 5. There is a 1:N relationship from GO_TERM to GO_TERM_RELATION (parent term role).
 - Each parent gene ontology term in a gene ontology term relationship may belong to zero or more relations (partial participation).
 - Each gene ontology term relation must contain exactly one parent gene ontology term (total participation).
 - This relationship is non-identifying.
- 6. There is a 1:N relationship from GO_TERM to GO_TERM_RELATION (child term role).
 - Each child gene ontology term in a gene ontology term relationship may belong to zero or more relations (partial participation).
 - Each gene ontology term relation must contain exactly one child gene ontology term (total participation).
 - This relationship is non-identifying.
- 7. There exists a M:N relationship from GENE_ANNOTATION to SOURCE as an annotation must contain one or more sources and a source may correspond to zero or more annotations. To resolve this many-to-many relationship, an intersection entity, ANNOTATION_SOURCE, is created.
 - There is a 1:N relationship from GENE_ANNOTATION to ANNOTATION_SOURCE.
 - Each gene annotation must contain one or more sources (total participation).
 - Each gene annotation source must belong to exactly one gene annotation (total participation).
 - This relationship is identifying as the ANNOTATION_SOURCE is a weak entity type and completely depends on its parents, GENE_ANNOTATION and SOURCE, to be identified.
 - There is a 1:N relationship from SOURCE to ANNOTATION_SOURCE.
 - Each source must correspond to one or more gene annotation sources (total participation).
 - Each gene annotation source must correspond to exactly one source (total participation).
 - This relationship is identifying as the ANNOTATION_SOURCE is a weak entity type and completely depends on its parents, GENE_ANNOTATION and SOURCE, to be identified.
- 8. There exists a M:N relationship from GENE to GENE_DISEASE as a gene may be associated with zero or more genetic diseases and a genetic disease must involve one or more genes. To resolve this many-to-many relationship, an intersection entity, GENE_DISEASE_ASSOCIATION, is created.
 - There is a 1:N relationship from GENE to GENE_DISEASE_ASSOCIATION.
 - Each gene may be associated with zero or more gene-disease associations (partial participation).
 - Each gene-disease association must contain exactly one gene (total participation).

- This relationship is identifying as the GENE_DISEASE_ASSOCIATION is a weak entity type and completely depends on its parents, GENE and GENE_DISEASE, to be identified.
- There is a 1:N relationship from GENE_DISEASE to GENE_DISEASE_ASSOCIATION.
 - Each genetic disease must belong to one or more gene-disease associations (total participation).
 - Each gene-disease association must contain exactly one genetic disease (total participation).
 - This relationship is identifying as the GENE_DISEASE_ASSOCIATION is a weak entity type and completely depends on its parents, GENE and GENE_DISEASE, to be identified.
- 9. There exists a M:N relationship from GENE to GENE_PATHWAY as a gene may be associated with zero or more gene pathways and a gene pathway must have one or more genes participating in it. To resolve this many-to-many relationship, an intersection entity, GENE_PATHWAY_ASSOCIATION, is created.
 - There is a 1:N relationship from GENE to GENE_PATHWAY_ASSOCIATION.
 - Each gene may be a part of zero or more gene-pathway associations (partial participation).
 - Each gene-pathway association must contain exactly one gene (total participation).
 - This relationship is identifying as the GENE_PATHWAY_ASSOCIATION is a weak entity type and completely depends on its parents, GENE and GENE_PATHWAY, to be identified.
 - There is a 1:N relationship from GENE_PATHWAY to GENE_PATHWAY_ASSOCIATION.
 - Each gene pathway must be a part of one or more gene-pathway associations (total participation).
 - Each gene-pathway association must contain exactly one gene pathway (total participation).
 - This relationship is identifying as the GENE_PATHWAY_ASSOCIATION is a weak entity type and completely depends on its parents, GENE and GENE_PATHWAY, to be identified.
- 10. There exists a M:N relationship from GENE to GENE_FAMILY as a gene may be a part of zero or more gene families and a gene family must contain one or more genes. To resolve this many-to-many relationship, an intersection entity, GENE_FAMILY_MEMBER, is created.
 - There is a 1:N relationship from GENE to GENE_FAMILY_MEMBER.
 - Each gene may be associated with zero or more gene family members. (partial participation).
 - Each gene family member must be associated with exactly one gene (total participation).
 - This relationship is identifying as the GENE_FAMILY_MEMBER is a weak entity type and completely depends on its parents, GENE and GENE_FAMILY, to be identified.
 - There is a 1:N relationship from GENE_FAMILY to GENE_FAMILY_MEMBER.
 - Each gene family must contain one or more gene family members (total participation).
 - Each gene family member must correspond to exactly one gene family (total participation).
 - This relationship is identifying as the GENE_FAMILY_MEMBER is a weak entity type and completely depends on its parents, GENE and GENE_FAMILY, to be identified.

3.2.3 ER Diagram

Shown below is the Entity Relationship Diagram for the Integrated Gene Knowledge Database (IGKD).



3.3 Relational Model

In this section, the data dictionary containing metadata for all attributes is presented. We will also outline the operational rules and database operations that describe how data is stored, accessed, and maintained.

3.3.1 Data Dictionary

Shown below are the detailed descriptions of the metadata for each entity.

GENE

Column Name	Description	Data Type	Size	Constraint Type	Not Null?	Valid Values
gene_id	uniquely identifies each gene	INT	N/A	Primary Key	Y	integer values
gene_symbol	unique abbreviation for a gene name	VARCHAR	20	Unique	Y	alphanumeric characters and hyphens
short_description	brief description of the gene	VARCHAR	500	N/A	N	any text
chromosome	name of the chromosome the gene belongs to	VARCHAR	20	N/A	Y	any text
start_position	starting base-pair position of a gene on a chromosome	INT	N/A	Check	Y	positive integer values
end_position	ending base-pair position of a gene on a chromosome	INT	N/A	Check	Y	positive integer values
strand	indicates if strand is forward or reverse	CHAR	1	Check	Y	'+' and '-'
organism_id	identifier of the organism to which the gene belongs to	INT	N/A	Foreign Key	Y	integer values

ORGANISM

Column Name	Description	Data Type	Size	Constraint Type	Not Null?	Valid Values
organism_id	uniquely identifies the organism	INT	N/A	Primary Key	Y	integer values
scientific_name	scientific name of the organism	VARCHAR	100	Unique	Y	scientific names following binomial nomenclature
domain	name of the domain the organism belongs to	VARCHAR	8	Check	Y	'bacteria', 'archaea', 'eukarya'
genome_size	number of base pairs in genome	BIGINT	N/A	N/A	N	integer values
chromosome_count	number of chromosomes	INT	N/A	N/A	N	integer values
taxonomy_id	identifier for an organism used by NCBI Taxonomy	INT	N/A	N/A	N	integer values

GENE_DISEASE

Column Name	Description	Data Type	Size	Constraint Type	Not Null?	Valid Values
disease_id	uniquely identifies the disease a gene may be a part of	INT	N/A	Primary Key	Y	integer values
disease_name	name of the disease a gene may be a part of	VARCHAR	150	Unique	Y	any text
category	name of the disease category	VARCHAR	15	Check	Y	'genetic', 'infectious', 'metabolic', 'cancer', 'cardiovascular', 'neurological', 'respiratory', 'autoimmune', 'other', 'unknown'

GENE_DISEASE_ASSOCIATION

Column Name	Description	Data Type	Size	Constraint Type	Not Null?	Valid Values
gene_id	identifier of the gene belonging to this disease	INT	N/A	Foreign Key, Primary Key	Y	integer values
disease_id	identifier of the disease this gene belongs to	INT	N/A	Foreign Key, Primary Key	Y	integer values

GENE_PATHWAY

Column Name	Description	Data Type	Size	Constraint Type	Not Null?	Valid Values
pathway_id	uniquely identifies the pathway a gene may belong to	INT	N/A	Primary Key	Y	integer values
pathway_name	name of the gene pathway	VARCHAR	150	Unique	Y	any text
description	describes the gene pathway	VARCHAR	500	N/A	N	any text

GENE_PATHWAY_ASSOCIATION

Column Name	Description	Data Type	Size	Constraint Type	Not Null?	Valid Values
gene_id	identifier of the gene belonging to this pathway	INT	N/A	Foreign Key, Primary Key	Y	integer values
pathway_id	identifier of the pathway this gene belongs to	INT	N/A	Foreign Key, Primary Key	Y	integer values

GENE FAMILY

Column Name	Description	Data Type	Size	Constraint Type	Not Null?	Valid Values
family_id	uniquely identifies the gene family a gene may be a part of	INT	N/A	Primary Key	Y	integer values
family_name	name of the gene family	VARCHAR	150	Unique	Y	any text
description	describes the gene family	VARCHAR	500	N/A	N	any text

GENE FAMILY MEMBER

Column Name	Description	Data Type	Size	Constraint Type	Not Null?	Valid Values
gene_id	identifier of the gene belonging to this gene family	INT	N/A	Foreign Key, Primary Key	Y	integer values
family_id	identifier of the family this gene belongs to	INT	N/A	Foreign Key, Primary Key	Y	integer values

GENE PRODUCT

Column Name	Description	Data Type	Size	Constraint Type	Not Null?	Valid Values
gene_product_id	uniquely identifies the gene product	INT	N/A	Primary Key	Y	integer values
product_name	name of the gene product (from NCBI)	VARCHAR	100	N/A	Y	any text
product_type	name of the general product type	VARCHAR	7	Check	Y	'dna', 'rna', 'protein', 'other'
description	description of the product type	VARCHAR	500	N/A	N	any text
gene_id	identifier of the gene to which the gene product belongs to	INT	N/A	Foreign Key	Y	integer values

Term Project Report: Integrated Gene Knowledge Database

GO TERM

Column Name	Description	Data Type	Size	Constraint Type	Not Null?	Valid Values
go_id	uniquely identifies the gene ontology term	VARCHAR	15	Primary Key, Check	Y	any GO term identifier in the format 'GO:0000000' where 0 is a placeholder for any digit
term_name	name of the gene ontology term	VARCHAR	200	Unique	Y	any text
ontology	name of the sub-ontology the term belongs to	VARCHAR	2	Check	Y	'cc', 'mf', 'bp'
definition	definition of gene ontology term	VARCHAR	500	N/A	Y	any text
date_created	date and time the gene ontology term was added to database	DATETIME	N/A	N/A	Y	any valid date in the YYYY-MM-DD format and time in HH:MM:SS format
date_last_update	date and time the gene ontology term was last updated	DATETIME	N/A	Check	Y	any valid date in the YYYY-MM-DD format and time in HH:MM:SS format

GO TERM RELATION

Column Name	Description	Data Type	Size	Constraint Type	Not Null?	Valid Values
go_relation_id	uniquely identifies the relation between two gene ontology terms	INT	N/A	Primary Key	Y	integer values
parent_go_id	identifier of the parent gene ontology term associated with this relation	VARCHAR	15	Foreign Key	Y	any GO term identifier in the format 'GO:0000000' where 0 is a placeholder for any digit
child_go_id	identifier of the child gene ontology term associated with this relation	VARCHAR	15	Foreign Key	Y	any GO term identifier in the format 'GO:0000000' where 0 is a placeholder for any digit
relation_term	describes the relationship between 2 gene ontology terms	VARCHAR	9	Check	Y	'is a', 'part of', 'has part', 'regulates', 'occurs in'

GENE_ANNOTATION

Column Name	Description	Data Type	Size	Constraint Type	Not Null?	Valid Values
annotation_id	uniquely identifies the gene annotation	INT	N/A	Primary Key	Y	integer values
gene_product_id	identifier of the gene product the gene ontology term is associated with	INT	N/A	Foreign Key, part of Unique(gene_product_id, go_id)	Y	integer values
go_id	identifier of the gene ontology term associated with this gene product	VARCHAR	15	Foreign Key, part of Unique(gene_product_id, go_id)	Y	any GO term identifier in the format 'GO:0000000' where 0 is a placeholder for any digit
evidence_code	indicates how the annotation of a term is supported	VARCHAR	3	Check	Y	'EXP', 'IDA', 'IPI', 'IMP', 'IGI', 'IEP', 'HTP', 'HDA', 'HMP', 'HGI', 'HEP', 'IBA', 'IBD', 'IKR', 'IRD', 'ISS', 'ISO', 'ISA', 'ISM', 'IGC', 'RCA', 'TAS', 'NAS', 'IC', 'ND', 'IEA'
date_created	date and time the annotation was added to the database	DATETIME	N/A	N/A	Y	any valid date in the YYYY-MM-DD format and time in HH:MM:SS format
date_last_update	date and time the annotation was last updated	DATETIME	N/A	Check	Y	any valid date in the YYYY-MM-DD format and time in HH:MM:SS format
comment	comments on the gene annotation	VARCHAR	1000	N/A	N	any text

SOURCE

Column Name	Description	Data Type	Size	Constraint Type	Not Null?	Valid Values
source_id	uniquely identifies the source	INT	N/A	Primary Key	Y	integer values
source_name	name of the source	VARCHAR	150	N/A	Y	any text
publish_year	year the source was published or released	YEAR	N/A	N/A	Y	any valid year
url	url linking to the source	VARCHAR	2048	N/A	Y	any text

ANNOTATION_SOURCE

Column Name	Description	Data Type	Size	Constraint Type	Not Null?	Valid Values
source_id	identifier of the source associated with this annotation	INT	N/A	Foreign Key, Primary Key	Y	integer values
annotation_id	identifier of annotation associated with this source	INT	N/A	Foreign Key, Primary Key	Y	integer values

3.3.2 Integrity Rules

The referential integrity constraints are established using foreign key constraints. Referential integrity is further supported from foreign key actions such as ON DELETE and ON UPDATE to control how changes in the parent table affect related child records. These are shown below:

1. GENE
 - The foreign key organism_id references the parent table ORGANISM.
 - ON DELETE RESTRICT
 - ON UPDATE CASCADE
2. GENE_PRODUCT
 - The foreign key gene_id references the parent table GENE.
 - ON DELETE RESTRICT
 - ON UPDATE CASCADE
3. GENE_ANNOTATION
 - The foreign key gene_product_id references the parent table GENE_PRODUCT.
 - ON DELETE RESTRICT
 - ON UPDATE CASCADE
 - The foreign key go_id references the parent table GO_TERM.
 - ON DELETE RESTRICT
 - ON UPDATE CASCADE
4. GO_TERM_RELATION
 - The foreign key parent_go_id references the parent table GO_TERM.
 - ON DELETE RESTRICT
 - ON UPDATE CASCADE
 - The foreign key child_go_id references the parent table GO_TERM.
 - ON DELETE RESTRICT
 - ON UPDATE CASCADE
5. ANNOTATION_SOURCE
 - The foreign key source_id references the parent table SOURCE.
 - ON DELETE CASCADE

- ON UPDATE CASCADE
 - The foreign key annotation_id references the parent table GENE_ANNOTATION.
 - ON DELETE CASCADE
 - ON UPDATE CASCADE
- 6. GENE_DISEASE_ASSOCIATION
 - The foreign key gene_id references the parent table GENE.
 - ON DELETE CASCADE
 - ON UPDATE CASCADE
 - The foreign key disease_id references the parent table GENE_DISEASE.
 - ON DELETE CASCADE
 - ON UPDATE CASCADE
- 7. GENE_PATHWAY_ASSOCIATION
 - The foreign key gene_id references the parent table GENE.
 - ON DELETE CASCADE
 - ON UPDATE CASCADE
 - The foreign key pathway_id references the parent table GENE_PATHWAY.
 - ON DELETE CASCADE
 - ON UPDATE CASCADE
- 8. GENE_FAMILY_MEMBER
 - The foreign key gene_id references the parent table GENE.
 - ON DELETE CASCADE
 - ON UPDATE CASCADE
 - The foreign key family_id references the parent table GENE_FAMILY.
 - ON DELETE CASCADE
 - ON UPDATE CASCADE

The ON DELETE RESTRICT action is used to prevent the deletion of a parent record if the child record it relates to still exists. This prevents accidental data loss and maintains referential integrity by requiring dependent records to be handled first. This action is found in entities where data consistency is important.

The ON DELETE CASCADE action is used to automatically remove child records when their corresponding parent record is deleted. This is found in associative tables where the child record has no meaning without the parent. This action helps maintain consistency and eliminates the need for manual cleanup of data.

The ON UPDATE CASCADE action propagates updates of primary key values in a parent table to the corresponding child tables. This action ensures that relationships are maintained when the primary key changes and helps maintain consistency across the database without the need for manual updates. In practice, there is little reason for all of the primary key values (except for go_id) to change, as they are surrogate keys that hold no meaning.

The mandatory fields are handled using the NOT NULL constraint.

Entity integrity is enforced through primary key constraints which are defined for each table. Every entity must contain a unique primary key that cannot be left null. This ensures that each record can be uniquely identified. A natural primary key cannot be used for most tables, so surrogate keys are used instead.

Domain constraints are enforced through the use of data types, which restrict the accepted values for each attribute, and CHECK constraints. The CHECK constraint further limits the range or sets of allowable values. Some examples are the attribute called 'strand' from GENE which restricts values to '+' or '-'. Another example is the 'start_position' attribute in GENE which restricts acceptable values to integer values greater than or equal to 0 and less than the value stored in 'end_position'. This ensures that only valid positional ranges are used. Additionally, the 'go_id' attribute in GO_TERM is constrained using a regular expression (^GO:[0-9]{7}\$) to follow the format standardized by the Gene Ontology Consortium.

3.3.3 Operational Rules

Listed below are some of the constraints for the database operations:

1. A gene cannot be deleted if it is referenced by any gene products. Instead, the gene products must be manually removed first before removing the gene.
2. An organism cannot be deleted if there is at least 1 gene associated with it. This is to prevent the accidental deletion of large amounts of data as each organism can contain thousands of genes. All of the genes must be removed first (can be removed in mass) before removing an organism.
3. A GO term cannot be deleted if it is involved in a relation. This is to let the user know that the GO term is still in use. The user will have to search for relations the GO term is a part of and delete those relations first in order to remove the GO term.
4. A GO term and gene product cannot be deleted if they are referenced in a gene annotation as the annotation requires both terms. If a user wants to delete a GO term or gene product, they must first remove the gene annotation.
5. A particular gene cannot be entered for an organism that doesn't exist.
6. A gene product cannot be added for a gene that does not exist.
7. A gene annotation cannot be added if the GO term and gene product it references do not exist.
8. A GO term relation cannot be added if the parent GO term or child GO term do not exist.
9. A source cannot be added without being attributed to at least 1 gene annotation. Similarly, a gene annotation cannot be added without at least referencing 1 source. This is to ensure that all annotations are supported by evidence.

3.3.4 Operations

In this section, we will describe the operations required for use in the Integrated Gene Knowledge Database.

1. Adding a Gene Annotation
 - This operation involves the retrieve and insert statements. When a researcher (this role is discussed in more detail in the section 3.4 Security) wants to add a gene annotation with an associated source, a retrieval is first performed to verify that both the specified gene product and gene ontology term exist. Once this is confirmed, a new record is inserted into the SOURCE table, and a new annotation is inserted into the GENE_ANNOTATION table. This process is handled by a stored procedure as the researcher does not have direct privileges to insert into the SOURCE table. The operation concludes with a relationship between the source and gene annotation being established in the ANNOTATION_SOURCE table.
2. Adding an Additional Source for a Gene Annotation
 - This operation involves the retrieve and insert statements. When a researcher wants to add a source for a gene annotation, a retrieval is first performed to verify that the annotation exists. Once this has been confirmed, a new record is inserted into the SOURCE table. This process is again handled by a stored procedure. The operation concludes with a relationship between the source and gene annotation being established in the ANNOTATION_SOURCE table.
3. Adding a GO Term
 - This operation involves the retrieve and insert statements. When a researcher wants to add a new gene ontology term, a retrieval is first performed to verify that the specified GO term ID is unique and does not already exist. Once this is confirmed, a new record can then be inserted into the GO_TERM table if the required information is provided.
4. Adding a Gene Product for a Gene
 - This operation involves the retrieve and insert statements. When a researcher wants to add a gene product, a retrieval is first performed to verify that the specified gene exists. A second retrieval is then performed to ensure that the gene product does not already exist for that particular gene. Once both of these conditions are satisfied, a new record can be inserted into the GENE_PRODUCT table if the required information is provided.
5. Adding a Gene for an Organism

- This operation involves the retrieve and insert statements. When a researcher wants to add a gene, a retrieval is first performed to verify that the specified organism exists. A second retrieval is then performed to ensure that a gene with the same gene symbol does not already exist for that organism. Once both of these conditions are satisfied, a new record can be inserted into the GENE table if the required information is provided.

3.4 Security

To maintain database security, roles and privileges are assigned to enforce control access, authentication is enforced through MySQL user accounts with password protection, and best practices are followed to prevent SQL injection. There are three roles in this database: database_manager, researcher, and analyst. The analyst has the least privileges as they only have a read-only role for querying and analyzing data.

Their privileges include:

- SELECT from all tables
- Access to all functions

The researcher carries on the same privileges as the analyst but has a few more. They are allowed to add data and make changes to existing data in the database. Their privileges include:

- INSERT on all tables except SOURCE
- UPDATE on all tables
- Access to all stored procedures

The database manager role has the most privileges and full access to the database. They have the same privileges as the analyst and researcher, as well as the following:

- DELETE on all tables

The researcher is not granted the DELETE privilege to prevent accidental data loss and preserve data integrity. If an error is made or they have a valid reason to remove data, they must request for the review and removal of said data to the database manager. This approach ensures that deletions are controlled and intentional.

Shown below is a screenshot showing the successful creation of the database_manager, researcher, and analyst roles and assignment of privileges. Three users have been created, as seen in lines 26-28, with a user for each role.

Term Project Report: Integrated Gene Knowledge Database

	Time	Action	Response
✓ 1	16:19:33	CREATE ROLE 'database_manager'	0 row(s) affected
✓ 2	16:19:33	CREATE ROLE 'researcher'	0 row(s) affected
✓ 3	16:19:33	CREATE ROLE 'analyst'	0 row(s) affected
✓ 4	16:19:33	GRANT SELECT, INSERT, UPDATE, DELETE, EXECUTE ON IGKD.* TO 'database_manager'	0 row(s) affected
✓ 5	16:19:33	GRANT SELECT, UPDATE, EXECUTE ON IGKD.* TO 'researcher'	0 row(s) affected
✓ 6	16:19:33	GRANT INSERT ON IGKD.GENE TO 'researcher'	0 row(s) affected
✓ 7	16:19:33	GRANT INSERT ON IGKD.ORGANISM TO 'researcher'	0 row(s) affected
✓ 8	16:19:33	GRANT INSERT ON IGKD.GENE_DISEASE TO 'researcher'	0 row(s) affected
✓ 9	16:19:33	GRANT INSERT ON IGKD.GENE_DISEASE_ASSOCIATION TO 'researcher'	0 row(s) affected
✓ 10	16:19:33	GRANT INSERT ON IGKD.GENE_FAMILY TO 'researcher'	0 row(s) affected
✓ 11	16:19:33	GRANT INSERT ON IGKD.GENE_FAMILY_MEMBER TO 'researcher'	0 row(s) affected
✓ 12	16:19:33	GRANT INSERT ON IGKD.GENE_PATHWAY TO 'researcher'	0 row(s) affected
✓ 13	16:19:33	GRANT INSERT ON IGKD.GENE_PATHWAY_ASSOCIATION TO 'researcher'	0 row(s) affected
✓ 14	16:19:33	GRANT INSERT ON IGKD.GENE_PRODUCT TO 'researcher'	0 row(s) affected
✓ 15	16:19:33	GRANT INSERT ON IGKD.GENE_ANNOTATION TO 'researcher'	0 row(s) affected
✓ 16	16:19:33	GRANT INSERT ON IGKD.GO_TERM TO 'researcher'	0 row(s) affected
✓ 17	16:19:33	GRANT INSERT ON IGKD.GO_TERM_RELATION TO 'researcher'	0 row(s) affected
✓ 18	16:19:33	GRANT INSERT ON IGKD.ANNOTATION_SOURCE TO 'researcher'	0 row(s) affected
✓ 19	16:19:33	GRANT SELECT ON IGKD.* TO 'analyst'	0 row(s) affected
✓ 20	16:19:33	GRANT EXECUTE ON FUNCTION IGKD.count_genes_organism TO 'analyst'	0 row(s) affected
✓ 21	16:19:33	GRANT EXECUTE ON FUNCTION IGKD.gene_annotation_count TO 'analyst'	0 row(s) affected
✓ 22	16:19:33	GRANT EXECUTE ON FUNCTION IGKD.gene_organism_count TO 'analyst'	0 row(s) affected
✓ 23	16:19:33	GRANT EXECUTE ON FUNCTION IGKD.retrieve_gene_id TO 'analyst'	0 row(s) affected
✓ 24	16:19:33	GRANT EXECUTE ON FUNCTION IGKD.retrieve_gene_product_id TO 'analyst'	0 row(s) affected
✓ 25	16:19:33	FLUSH PRIVILEGES	0 row(s) affected
✓ 26	16:19:33	CREATE USER 'user1'@'localhost' IDENTIFIED BY 'pass1' PASSWORD EXPIRE	0 row(s) affected
✓ 27	16:19:33	CREATE USER 'user2'@'localhost' IDENTIFIED BY 'pass2' PASSWORD EXPIRE	0 row(s) affected
✓ 28	16:19:33	CREATE USER 'user3'@'localhost' IDENTIFIED BY 'pass3' PASSWORD EXPIRE	0 row(s) affected
✓ 29	16:19:33	GRANT 'database_manager' TO 'user1'@'localhost'	0 row(s) affected
✓ 30	16:19:33	GRANT 'researcher' TO 'user2'@'localhost'	0 row(s) affected
✓ 31	16:19:33	GRANT 'analyst' TO 'user3'@'localhost'	0 row(s) affected
✓ 32	16:19:33	SET DEFAULT ROLE 'database_manager' TO 'user1'@'localhost'	0 row(s) affected
✓ 33	16:19:33	SET DEFAULT ROLE 'researcher' TO 'user2'@'localhost'	0 row(s) affected
✓ 34	16:19:33	SET DEFAULT ROLE 'analyst' TO 'user3'@'localhost'	0 row(s) affected

We will log in to the database as 'user3' who has the analyst role to demonstrate their privileges. The screenshot below confirms that this user has the analyst role.

105	SHOW GRANTS for 'user3'@'localhost';
100%	37:105
Result Grid	Filter Rows: Search Export:
Grants for user3@localhost	
GRANT USAGE ON *.* TO `user3`@`localhost`	
GRANT `analyst` @`%` TO `user3`@`localhost`	

Before we can demonstrate the privileges, we need to create a new connection called 'user3_test' with the username 'user3' and old password 'pass3' (this has been updated to a new password). The screenshot below shows the connection.

Term Project Report: Integrated Gene Knowledge Database

The screenshot shows the MySQL Workbench connection configuration window for a connection named 'user3_test'. The 'Connection' tab is selected. The 'Connection Method' is set to 'Standard (TCP/IP)'. The 'Parameters' sub-tab is active, showing the following fields: Hostname (127.0.0.1), Port (3306), Username (user3), Password (with 'Store in Keychain ...' and 'Clear' buttons), and Default Schema (IGKD). To the right of these fields are descriptive labels: 'Name or IP address of the server host - and TCP/IP port.', 'Name of the user to connect with.', 'The user's password. Will be requested later if it's not set.', and 'The schema to use as default schema. Leave blank to select it later.'

To demonstrate that the privileges for the analyst role work properly, we will perform a SELECT command on the GENE table, use the gene_organism_count function, and try to INSERT a record into the GENE_FAMILY table.

The screenshot shows the MySQL Workbench SQL editor with the following SQL commands:

```
1 • SELECT * FROM GENE;  
2  
3 • SELECT gene_organism_count('Homo sapiens');  
4  
5 • INSERT INTO GENE_FAMILY (family_name)  
6   VALUES ('globin gene family');
```

Below the editor is the 'Action Output' window, which displays the results of the executed commands:

	Time	Action	Response
✓ 1	16:31:43	SELECT * FROM GENE LIMIT 0, 1000	29 row(s) returned
✓ 2	16:31:43	SELECT gene_organism_count('Homo sapiens') LIMIT 0, 1000	1 row(s) returned
✗ 3	16:31:43	INSERT INTO GENE_FAMILY (family_name) VALUES ('globin gene family')	Error Code: 1142. INSERT command denied to user 'user3'@'localhost' for table 'gene_family'

As we can see in the screenshot above, the user was able to perform the first two operations without issue, but we received an error when trying to insert a record into a GENE_FAMILY. The error says that the command was denied and this is because the analyst role does not have the privilege to use INSERT on any tables.

We reduce the risk of SQL injection by separating user input from SQL logic and enforcing the use of controlled parameters. This is achieved by handling many insert operations through stored procedures. Furthermore, parameterized variables are used in SQL queries, where the user input is stored in variables and strictly treated as data instead of being concatenated into dynamic SQL statements.

3.5 Database Backup and Recovery

Database backup of the entire database is implemented using the mysqldump command in macOS terminal. The command used is shown in the screenshot below.

```
Anishs-MacBook-Air-6:~ anish$ mysqldump -u root -p IGKD > IGKD_backup.sql  
Enter password:  
Anishs-MacBook-Air-6:~ anish$
```

The backup file contains the complete database schema and data. This includes the table structures, records, stored procedures, functions, and triggers. The generated .sql file can be opened in MySQL Workbench and executed as a SQL script to restore the database. The screenshot below shows a portion of the file contents. We can see the DDL statements (CREATE TABLE and ALTER TABLE) used to define the

ANNOTATION_SOURCE table, as well as the DML statements (INSERT INTO) used to populate the table with data.

```

1  -- MySQL dump 10.13  Distrib 8.0.45, for macos26.3 (arm64)
2  --
3  -- Host: localhost    Database: IGKD
4  --
5  -- Server version      8.0.45
6
7  /*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
8  /*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
9  /*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
10 /*!50503 SET NAMES utf8mb4 */;
11 /*!40103 SET @OLD_TIME_ZONE=@@TIME_ZONE */;
12 /*!40103 SET TIME_ZONE='+00:00' */;
13 /*!40014 SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0 */;
14 /*!40014 SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0 */;
15 /*!40101 SET @OLD_SQL_MODE=@@SQL_MODE, SQL_MODE='NO_AUTO_VALUE_ON_ZERO' */;
16 /*!40111 SET @OLD_SQL_NOTES=@@SQL_NOTES, SQL_NOTES=0 */;
17
18 --
19 -- Table structure for table `ANNOTATION_SOURCE`
20 --
21
22 * DROP TABLE IF EXISTS `ANNOTATION_SOURCE`;
23 * /*!40101 SET @saved_cs_client      = @@character_set_client */;
24 * /*!50503 SET character_set_client = utf8mb4 */;
25 * CREATE TABLE `ANNOTATION_SOURCE` (
26   `source_id` int NOT NULL,
27   `annotation_id` int NOT NULL,
28   PRIMARY KEY (`source_id`,`annotation_id`),
29   KEY `ANNSOURCE_ANNOTATION_FK` (`annotation_id`),
30   CONSTRAINT `ANNSOURCE_ANNOTATION_FK` FOREIGN KEY (`annotation_id`) REFERENCES `GENE_ANNOTATION` (`annotation_id`) ON DELETE CASCADE ON UPDATE CASCADE,
31   CONSTRAINT `ANNSOURCE_SOURCE_FK` FOREIGN KEY (`source_id`) REFERENCES `SOURCE` (`source_id`) ON DELETE CASCADE ON UPDATE CASCADE
32 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
33 * /*!40101 SET character_set_client = @saved_cs_client */;
34
35 --
36 -- Dumping data for table `ANNOTATION_SOURCE`
37 --
38
39 * LOCK TABLES `ANNOTATION_SOURCE` WRITE;
40 * /*!40000 ALTER TABLE `ANNOTATION_SOURCE` DISABLE KEYS */;
41 * INSERT INTO `ANNOTATION_SOURCE` VALUES (1,1),(2,2),(3,3),(5,5),(6,6);
42 * /*!40000 ALTER TABLE `ANNOTATION_SOURCE` ENABLE KEYS */;
43 * UNLOCK TABLES;

```

3.6 Using Database Design or CASE Tool

The primary CASE (Computer-Aided Software Engineering) tool used in this project was MySQL Workbench. Its built-in Entity-Relationship diagram tool was used to design the schema of the database, including the entities, attributes, and relationships. The SQL editor and graphical interface were used to create and modify tables, write queries, and implement advanced features such as stored procedures, functions, and triggers, as well as to test and manage the database.

3.7 Other Possible ER Relationships

During the design of this database, a few alternative approaches were considered to represent the data.

For example, I initially planned to include an entity called PRODUCT_TYPE to store the different types of gene products. This entity would include attributes such as subtype, molecule_type, and description. The entity would have a 1:N relationship with gene product, where a gene product must belong to exactly one product type, but a product type can be associated with multiple gene products. This design was removed because the functionality it provided was limited and largely redundant. Similar information could be captured through gene annotations, which utilize GO terms and gene product terms; the comment attribute in GENE_ANNOTATION would allow researchers to discuss the gene product in further detail. As a result, the PRODUCT_TYPE entity was simplified into a product_type attribute within the GENE_PRODUCT table.

Another alternative design involved creating a recursive relationship within the GENE entity to capture the interactions between genes, such as inhibition. This approach was not implemented because the relationship can be modeled by existing entities. In many cases, gene interactions occur through their gene products, and this can be described using gene annotations. Furthermore, genes that interact with each other are often

part of the same biological pathway, making the GENE_PATHWAY entity more appropriate for representing these relationships.

Ultimately, design choices that introduced unnecessary complexity or redundancy without providing significant functional benefit were avoided in favor of a more efficient schema.

4. Implementation Description

In this section, we will cover the general database implementation, which includes the data dictionary, advanced features, and queries. We will describe how these implemented features support the overall functionality of the database.

4.1 Data Dictionary

The data dictionary is found in the pdf titled “Data_Dictionary.pdf” which has been attached to the submission. The results were obtained by running the following query and exporting to an Excel file:

```
1  SELECT
2      TABLE_NAME,
3      COLUMN_NAME,
4      COLUMN_TYPE,
5      IS_NULLABLE,
6      COLUMN_KEY,
7      EXTRA
8  FROM INFORMATION_SCHEMA.COLUMNS
9  WHERE TABLE_SCHEMA = 'IGKD'
10 ORDER BY TABLE_NAME, ORDINAL_POSITION;
```

4.2 Advanced Features

The Integrated Gene Knowledge Database makes use of 3 triggers, 4 functions, and 5 stored procedures to implement the rules expected of a bioinformatics database. It is important to note that ‘Safe Update Mode’ must be disabled for Trigger 3 to work.

Triggers:

1. before_go_term_update
2. before_gene_annotation_update
3. after_gene_annotation_delete

Functions:

1. retrieve_gene_id
2. gene_annotation_count
3. gene_organism_count
4. retrieve_gene_product_id

Stored Procedures:

1. add_annotation_with_source
2. add_additional_source
3. add_go_term
4. add_gene_product_for_gene
5. add_gene_for_organism

Trigger 1: before_go_term_update

Term Project Report: Integrated Gene Knowledge Database

This trigger occurs before a user updates the GO_TERM table. The trigger updates the date_last_update to reflect the current date and time. While simple, this trigger is important for keeping track of changes to a gene ontology term over time and ensures that the database is up to date. Shown below is a screenshot of the trigger:

```
1 • CREATE DEFINER = CURRENT_USER TRIGGER `IGKD`.`before_go_term_update` BEFORE UPDATE ON `GO_TERM` FOR EACH ROW
2 BEGIN
3     SET NEW.date_last_update = NOW();
4 END;
```

Now we will show a demonstration of the trigger going into effect. In this example, we want to update the definition of the gene ontology term corresponding to 'nucleus' within the GO_TERM table. First, we will show the most recent date_last_update value. As seen below, the value is '2026-04-09 16:38:52'.

```
1 • SELECT *
2 FROM GO_TERM
3 WHERE go_id = 'GO:0005737';
```

100% 28:3

Result Grid Filter Rows: Search Edit: Export/Import:

go_id	term_name	ontology	definition	date_created	date_last_update
GO:0005737	cytoplasm	cc	The jelly-like substance found inside a cell	2026-04-09 16:38:52	2026-04-09 16:38:52
NULL	NULL	NULL	NULL	NULL	NULL

After updating the definition attribute of the table, we see below that the date_last_update has also been updated automatically because of the trigger. It now shows '2026-04-09 16:42:38'.

```
1 • UPDATE GO_TERM
2 SET definition = 'The fluid-like substance that fills the interior of the cell'
3 WHERE go_id = 'GO:0005737';
4
5 • SELECT *
6 FROM GO_TERM
7 WHERE go_id = 'GO:0005737';
```

100% 28:7

Result Grid Filter Rows: Search Edit: Export/Import:

go_id	term_name	ontology	definition	date_created	date_last_update
GO:0005737	cytoplasm	cc	The fluid-like substance that fills the interior...	2026-04-09 16:38:52	2026-04-09 16:42:38
NULL	NULL	NULL	NULL	NULL	NULL

Trigger 2: before_gene_annotation_update

This trigger occurs before a user updates the GENE_ANNOTATION table. The trigger works the same as before_go_term_update; it updates the date_last_update to reflect the current date and time. Shown below is a screenshot of the trigger:

Term Project Report: Integrated Gene Knowledge Database

```
1 • CREATE DEFINER = CURRENT_USER TRIGGER `IGKD`.`before_gene_annotation_update` BEFORE UPDATE ON `GENE_ANNOTATION` FOR EACH ROW
2 BEGIN
3     SET NEW.date_last_update = NOW();
4 END;
```

Now we will show a demonstration of the trigger going into effect. In this example, we want to update the comment of the gene annotation with annotation_id = 5 within the GENE_ANNOTATION table. First, we will show the most recent date_last_update value. As seen below, the value is '2026-04-09 16:38:52'.

```
1 • SELECT *
2 FROM GENE_ANNOTATION
3 WHERE annotation_id = 5;
```

100% 25:3

Result Grid Filter Rows: Search Edit: Export/Import:

annotation_id	gene_product...	go_id	evidence_code	date_created	date_last_update	comment
5	8	GO:0005634	EXP	2026-04-09 16:38:52	2026-04-09 16:38:52	FLC functions as a nuclear transcription factor
NULL	NULL	NULL	NULL	NULL	NULL	NULL

After updating the comment attribute of the table, we see below that the date_last_update has also been updated automatically because of the trigger. It now shows '2026-04-09 16:56:58'.

```
1 • UPDATE GENE_ANNOTATION
2 SET comment = 'FLC functions as a nuclear transcription factor that represses
3 flowering by regulating the expression of key floral transition genes'
4 WHERE annotation_id = 5;
5
6 • SELECT *
7 FROM GENE_ANNOTATION
8 WHERE annotation_id = 5;
```

100% 25:8

Result Grid Filter Rows: Search Edit: Export/Import:

annotation_id	gene_product...	go_id	evidence_code	date_created	date_last_update	comment
5	8	GO:0005634	EXP	2026-04-09 16:38:52	2026-04-09 16:56:58	FLC functions as a nuclear transcription factor that repr...
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Trigger 3: after_gene_annotation_delete

This trigger occurs after a record is deleted from GENE_ANNOTATION. Due to the ANNSOURCE_ANNOTATION_FK constraint, the corresponding record in ANNOTATION_SOURCE is also deleted. Since a trigger does not initiate on a child table as a result of a CASCADE action on a parent table, the trigger is added to the GENE_ANNOTATION table instead of the ANNOTATION_SOURCE table. The trigger starts by checking if the source corresponding to the deleted gene annotation is associated with any other gene annotations. If it isn't, then the source is removed from the database. Shown below is a screenshot of the trigger:

Term Project Report: Integrated Gene Knowledge Database

```
1 • CREATE DEFINER = CURRENT_USER TRIGGER `IGKD`.`after_gene_annotation_delete` AFTER DELETE ON `GENE_ANNOTATION` FOR EACH ROW
2 BEGIN
3     /* Checks if the deleted annotation's source_id is used anywhere else. If it isn't, then the source is removed */
4     DELETE FROM SOURCE
5     WHERE NOT EXISTS (
6         SELECT 1
7         FROM ANNOTATION_SOURCE a
8         WHERE a.source_id = SOURCE.source_id
9     );
10 END;
```

Now we will show a demonstration of the trigger going into effect. In this example, we will remove a gene annotation and check to see if its respective source is deleted as well. First, let's verify the source_id corresponding to annotation_id = 9.

```
1 • SELECT annotation_id, source_id
2 FROM ANNOTATION_SOURCE
3 WHERE annotation_id = 9;
```

100% 25:3

Result Grid Filter Rows: Search

annotation_id	source_id
9	8
HULL	HULL

As we can see above, the source_id is 8. We can check the SOURCE table and identify this record for extra verification.

```
1 • SELECT *
2 FROM SOURCE
3 WHERE source_id = 8;
```

100% 20:3

Result Grid Filter Rows: Search Edit: Export/Import:

source_id	source_name	publish_year	url
8	Interaction with plant transcription factors can m...	2012	https://www.pnas.org/doi/10.1073/pnas.1120764...
HULL	HULL	HULL	HULL

We will also show that the source with source_id = 8 is not associated with any other gene annotation.

1	•	SELECT	annotation_id, source_id
2		FROM	ANNOTATION_SOURCE
3		WHERE	source_id = 8;

100%	↕	23:2
Result Grid   Filter Rows: Search		
annotation_id	source_id	
9	8	
		

Now that we have confirmed that the source only belongs to one gene annotation, we will delete the gene annotation with annotation_id = 9 which will also delete the record from ANNOTATION_SOURCE as well. The screenshot below confirms that the record has been deleted in ANNOTATION_SOURCE.

1	•	DELETE FROM	GENE_ANNOTATION
2		WHERE	annotation_id = 9;
3			
4	•	SELECT	annotation_id, source_id
5		FROM	ANNOTATION_SOURCE
6		WHERE	annotation_id = 9;

100%	↕	25:6
Result Grid   Filter Rows: Search		
annotation_id	source_id	
		

Now, we will verify that the source has been removed from the SOURCE table. It is important to note that safe update mode must be disabled for this trigger to work.

1

•

SELECT *

2

FROM SOURCE

3

WHERE source_id = 8;

100%

↕

20:3

Result Grid





Filter Rows:

Search

source_id

source_name

publish_year

url

NULL

NULL

NULL

NULL

The record corresponding to the source_id of 8 has been successfully removed as it is no longer associated with any gene annotation.

Function 1: retrieve_gene_id

This function takes in two inputs: geneSymbol, which has the datatype VARCHAR(20), and scientificName, which was the datatype VARCHAR(100). It then retrieves the gene_id associated with the provided gene symbol and scientific name. If the organism (with the scientific name) doesn't exist or the gene symbol doesn't exist for that organism, a value of -1 is returned. Shown below is a screenshot of the function:

```

1  CREATE DEFINER=`root`@`localhost` FUNCTION `retrieve_gene_id` (
2      geneSymbol VARCHAR(20),
3      scientificName VARCHAR(100)
4  ) RETURNS int
5      READS SQL DATA
6  BEGIN
7      DECLARE result INT;
8      SELECT g.gene_id
9      INTO result
10     FROM GENE g
11     JOIN ORGANISM o
12         ON o.organism_id = g.organism_id
13     WHERE g.gene_symbol = geneSymbol
14         AND o.scientific_name = scientificName
15     LIMIT 1;
16     RETURN IFNULL(result, -1);
17 END

```

Now we will show a demonstration of the function going into effect. In this example, we want to retrieve the gene_id that corresponds to the gene_symbol 'EGFR' and scientific_name 'Homo sapiens'. The gene_symbol was selected randomly from the table below. We can see that this gene_symbol is associated with an organism_id of 1 which corresponds to the scientific_name 'Homo sapiens' (as can be seen in the second screenshot below).

```

1  SELECT *
2  FROM GENE
3  LIMIT 3;

```

	gene_id	gene_symbol	short_descripti...	chromosome	start_positi...	end_positi...	strand	organism_id
1		TP53	NULL	17	7668421	7687490	-	1
2		EGFR	NULL	7	55019017	55211628	+	1
3		APOE	NULL	19	44905796	44909393	+	1
	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

```

1  SELECT *
2  FROM ORGANISM
3  WHERE organism_id = 1;

```

100% 23:3

Result Grid Filter Rows: Search Edit: Export

	organism_id	scientific_name	domain	genome_size	chromosome_co...	taxonomy_id
1	1	Homo sapiens	eukarya	3200000000	46	9606
	NULL	NULL	NULL	NULL	NULL	NULL

We expect to get a gene_id value of 2 which is what we see in the screenshot below.

```

1 • SELECT retrieve_gene_id('EGFR', 'Homo sapiens');

```

100% 49:1

Result Grid Filter Rows: Search Export:

	retrieve_gene_id('EGFR', 'Homo sapie...
2	

If we provide a gene_symbol that doesn't exist (like 'EGFR999') we expect to get a value of -1, which is what we see in the screenshot below.

```

1 • SELECT retrieve_gene_id('EGFR999', 'Homo sapiens');

```

100% 33:1

Result Grid Filter Rows: Search Export:

	retrieve_gene_id('EGFR999', 'Homo sapie...
-1	

Function 2: gene_annotation_count

This function takes in two inputs: geneSymbol, which has the datatype VARCHAR(20), and scientificName, which was the datatype VARCHAR(100). It identifies the gene_id from the two inputs then counts the total number of annotations associated with that gene. Shown below is a screenshot of the function:

```

1 • CREATE DEFINER=`root`@`localhost` FUNCTION `gene_annotation_count` (
2     geneSymbol VARCHAR(20),
3     scientificName VARCHAR(100)
4 ) RETURNS int
5     READS SQL DATA
6 BEGIN
7     DECLARE geneID INT;
8     DECLARE result INT;
9
10    SELECT g.gene_id
11    INTO geneID
12    FROM GENE g
13    JOIN ORGANISM o
14        ON o.organism_id = g.organism_id
15    WHERE g.gene_symbol = geneSymbol
16        AND o.scientific_name = scientificName
17    LIMIT 1;
18
19    IF geneID IS NULL THEN
20        RETURN -1;
21    END IF;
22
23    SELECT COUNT(*)
24    INTO result
25    FROM GENE_PRODUCT gp
26    JOIN GENE_ANNOTATION ga
27        ON gp.gene_product_id = ga.gene_product_id
28    WHERE gp.gene_id = geneID;
29    RETURN result;
30 END

```

Now we will show a demonstration of the function going into effect. In this example, we want to retrieve the total number of annotations associated with gene_symbol 'TNF' and scientific_name 'Homo sapiens' (this is associated with gene_id = 4). As we can see in the screenshot below, each gene (including gene with gene_id = 4) only contains one annotation. Thus, we expect the function to return a value of 1.

1	SELECT	ga.annotation_id, gp.gene_id
2	FROM	GENE_ANNOTATION ga
3	JOIN	GENE_PRODUCT gp
4	ON	ga.gene_product_id = gp.gene_product_id;

100%	24:2
------	------

Result Grid	Filter Rows:	Search	Export:
-------------	--------------	--------	---------

	annotation_id	gene_id
1	4	
2	10	
3	20	
5	16	
6	17	

After calling the function, which can be seen below, we get the expected value of 1.

1	SELECT	gene_annotation_count('TNF', 'Homo sapiens');
---	--------	---

100%	53:1
------	------

Result Grid	Filter Rows:	Search	Export:
-------------	--------------	--------	---------

gene_annotation_count('TNF', 'Homo sapi...
1

If there is no gene annotations associated with a gene, a value of 0 is returned. But if a gene doesn't correspond to the gene symbol and scientific name provided, a value of -1 is returned (as can be seen below).

1	SELECT	gene_annotation_count('TNFF', 'Homo sapiens');
---	--------	--

100%	35:1
------	------

Result Grid	Filter Rows:	Search	Export:
-------------	--------------	--------	---------

gene_annotation_count('TNFF', 'Homo sapi...
-1

Function 3: gene_organism_count

This function takes in one input: scientificName, which has the datatype VARCHAR(100). It counts the total number of genes associated with an organism (identified by its scientific name). Shown below is a screenshot of the function:

```

1 • ○ CREATE DEFINER='root'@'localhost' FUNCTION `gene_organism_count` (
2     scientificName VARCHAR(100)
3 ) RETURNS int
4     READS SQL DATA
5 ○ BEGIN
6     DECLARE organismID INT;
7     DECLARE result INT;
8
9     SELECT organism_id
10    INTO organismID
11    FROM ORGANISM o
12    WHERE o.scientific_name = scientificName
13    LIMIT 1;
14
15 ○ IF organismID IS NULL THEN
16     RETURN -1;
17 ~ END IF;
18
19     SELECT COUNT(*)
20    INTO result
21    FROM GENE g
22    WHERE g.organism_id = organismID;
23
24     RETURN result;
25 END

```

Now we will show a demonstration of the function going into effect. In this example, we want to retrieve the total number of genes associated with the organism of scientific name 'Mus musculus'. If no such organism exists, a value of -1 is returned. We can run the query to count the total number of genes. Looking at the results below, we expect a value of 3 to be returned by the function.

1	•	SELECT COUNT(*)
2		FROM GENE g
3		JOIN ORGANISM o
4		ON g.organism_id = o.organism_id
5		WHERE o.scientific_name = 'Mus musculus';

100%	16:1
Result Grid	Filter Rows: Search
Export:	

COUNT(*)	
3	

When the function is called, we get a value of 3 as shown by the screenshot below.

1	•	SELECT gene_organism_count('Mus musculus');
---	---	---

100%	44:1
Result Grid	Filter Rows: Search
Export:	

gene_organism_count('Mus muscul...	
3	

If we provide the scientific name of an organism that doesn't exist or hasn't been added to the database, we get a value of -1 as shown below.

1	•	SELECT gene_organism_count('Humans');
---	---	---------------------------------------

100%	35:1
Result Grid	Filter Rows: Search
Export:	

gene_organism_count('Huma...	
-1	

Function 3: retrieve_gene_product_id

This function takes in three inputs: `gene_symbol1`, which has the datatype `VARCHAR(20)`, `scientific_name1`, which has the datatype `VARCHAR(100)`, and `product_name1`, which has the datatype `VARCHAR(100)`. The function retrieves the `gene_product_id` given the gene symbol, scientific name of the organism, and gene product name. Shown below is a screenshot of the function:

```

1 • ⊖ CREATE DEFINER='root'@'localhost' FUNCTION `retrieve_gene_product_id` (
2     gene_symbol1 VARCHAR(20),
3     scientific_name1 VARCHAR(100),
4     product_name1 VARCHAR(100)
5 ) RETURNS int
6     READS SQL DATA
7 ⊖ BEGIN
8     DECLARE result INT;
9     SELECT gp.gene_product_id
10    INTO result
11   FROM GENE_PRODUCT gp
12  JOIN GENE g
13     ON g.gene_id = gp.gene_id
14  JOIN ORGANISM o
15     ON g.organism_id = o.organism_id
16  WHERE g.gene_symbol = gene_symbol1
17         AND o.scientific_name = scientific_name1
18         AND gp.product_name = product_name1
19     LIMIT 1;
20     RETURN IFNULL(result, -1);
21 END

```

Now we will show a demonstration of the function going into effect. In this example, we want to retrieve the `gene_product_id` of the gene with gene symbol 'TNF', scientific name = 'Homo sapiens', and product name 'tumor necrosis factor'. The expected result of 1 is shown below.

Term Project Report: Integrated Gene Knowledge Database

```
1 SELECT o.scientific_name, g.gene_symbol, gp.product_name, gp.gene_product_id
2 FROM GENE_PRODUCT gp
3 JOIN GENE g
4 ON gp.gene_id = g.gene_id
5 JOIN ORGANISM o
6 ON g.organism_id = o.organism_id;
```

100% 40:1

Result Grid Filter Rows: Search Export:

	scientific_name	gene_symbol	product_name	gene_product_id
	Homo sapiens	TNF	tumor necrosis factor	1
	Homo sapiens	ACE	ACE protein	2
	Escherichia coli	rho	Rho protein	3
	Homo sapiens	BRCA1	BRCA1 protein	4
	Homo sapiens	BRCA2	BRCA2 protein	5
	Homo sapiens	PALB2	PALB2 protein	6
	Arabidopsis thaliana	PHYB	phytochrome B	7
	Arabidopsis thaliana	FLC	FLC protein	8
	Arabidopsis thaliana	PHYA	phytochrome A	9

The screenshot below shows the function being called and we get a value of 1 as expected.

```
1 • SELECT retrieve_gene_product_id('TNF', 'Homo sapiens', 'tumor necrosis factor');
```

100% 81:1

Result Grid Filter Rows: Search Export:

	retrieve_gene_product_id('TNF', 'Homo sapiens', 'tumor necrosis factor')
	1

If we change one of the inputs to a value that doesn't exist, we will get a value of -1. This is shown below:

```
1 • SELECT retrieve_gene_product_id('TNF', 'Homo sapiens', 'nonsensical factor');
```

100% 68:1

Result Grid Filter Rows: Search Export:

	retrieve_gene_product_id('TNF', 'Homo sapi...
	-1

Stored Procedure 1: add_annotation_with_source

This stored procedure allows a user to enter a gene annotation and its source in one simple procedure; a record in GENE_ANNOTATION is created and a record in SOURCE is created. The procedure takes in three inputs: go_id1 of datatype VARCHAR(15), gene_product_id1 of datatype INT, evidence_code1 of datatype VARCHAR(3), source_name1 of datatype VARCHAR(150), publish_year1 of datatype YEAR, and url1 of datatype VARCHAR(2048). Shown below is the procedure:


```

1 ● CREATE PROCEDURE `add_annotation_with_source` (
2     /* Name of inputs */
3     IN go_id1 VARCHAR(15),
4     IN gene_product_id1 INT,
5     IN evidence_code1 VARCHAR(3),
6     IN source_name1 VARCHAR(150),
7     IN publish_year1 YEAR,
8     IN url1 VARCHAR(2048))
9 BEGIN
10     DECLARE use_source_id INT;
11     DECLARE use_annotation_id INT;
12     /* Check if the gene product exists */
13     IF NOT EXISTS (
14         SELECT 1
15         FROM GENE_PRODUCT gp
16         WHERE gp.gene_product_id = gene_product_id1
17     ) THEN
18         SIGNAL SQLSTATE '45000'
19         SET MESSAGE_TEXT = 'Gene product does not exists';
20     END IF;
21     /* Check if the gene ontology term exists */
22     IF NOT EXISTS (
23         SELECT 1
24         FROM GO_TERM gt
25         WHERE gt.go_id = go_id1
26     ) THEN
27         SIGNAL SQLSTATE '45000'
28         SET MESSAGE_TEXT = 'Gene ontology term does not exist';
29     END IF;
30     /* We insert the source first then store the source_id */
31     INSERT INTO SOURCE (source_name, publish_year, url)
32     VALUES (source_name1, publish_year1, url1);
33     SET use_source_id = LAST_INSERT_ID();
34     /*We insert the gene annotation*/
35     INSERT INTO GENE_ANNOTATION (gene_product_id, go_id, evidence_code, date_created, date_last_update)
36     VALUES (gene_product_id1, go_id1, evidence_code1, NOW(), NOW());
37     SET use_annotation_id = LAST_INSERT_ID();
38     /* We insert the annotation_id and source_id into the associative table linking the two entities */
39     INSERT INTO ANNOTATION_SOURCE (source_id, annotation_id)
40     VALUES (use_source_id, use_annotation_id);
41 END

```

Now we will show a demonstration of the procedure going into effect. We will add a gene annotation with acceptable input values. This is only for demonstration purposes; the values aren't accurate and the inserted record will be later removed from the database. It is important to note that the gene ontology term and gene product must already exist. If they don't exist, an error message is displayed. The GENE_ANNOTATION and SOURCE table before and after the procedure is called will be shown. Only the last row of each table needs to be seen to verify if the procedure ran successfully.

GENE_ANNOTATION before procedure is called:

Term Project Report: Integrated Gene Knowledge Database

6 • **SELECT** *

7 **FROM** GENE_ANNOTATION;

100% 22:7

Result Grid Filter Rows: Search Edit: Export/Import:

	annotation_id	gene_product_id	go_id	evidence_code	date_created	date_last_update	comment
1	1		GO:0006915	IDA	2026-04-09 16:38:52	2026-04-09 16:38:52	TNF induces apoptosis via TNFR1 signaling
2	2		GO:0007165	IDA	2026-04-09 16:38:52	2026-04-09 16:38:52	ACE catalyzes angiotensin I to II, which activates signa...
3	3		GO:0009987	IDA	2026-04-09 16:38:52	2026-04-09 16:38:52	Rho protein mediates the termination of transcription in...
5	8		GO:0005634	EXP	2026-04-09 16:38:52	2026-04-09 16:56:58	FLC functions as a nuclear transcription factor that repr...
6	9		GO:0005737	IEA	2026-04-09 16:38:52	2026-04-09 16:38:52	PHYA is localized in the cytoplasm prior to being activa...
	NULL	NULL	NULL	NULL	NULL	NULL	NULL

SOURCE before procedure is called:

3 • **SELECT** *

4 **FROM** SOURCE;

100% 13:4

Result Grid Filter Rows: Search Edit: Export/Import:

	source_id	source_name	publish_year	url
1		Reactive Oxygen Species in TNF α -Induced Sig...	2010	https://www.sciencedirect.com/science/article/pii...
2		Physiology, Renin Angiotensin System	2026	https://www.ncbi.nlm.nih.gov/books/NBK470410/
3		Regulation of Rho-Dependent Transcription Ter...	2000	https://pubs.acs.org/doi/10.1021/bi992658z
5		Light-Regulated Nuclear Import and Degradatio...	2010	https://pmc.ncbi.nlm.nih.gov/articles/PMC30370...
6		UniProt	2024	https://www.uniprot.org/uniprotkb/Q9S7Z8/entry
	NULL	NULL	NULL	NULL

SOURCE table after procedure is called:

1 **call** add_annotation_with_source('GO:0007267', 1, 'IMP', 'Source', 2000, 'https://example.com');

2

3 • **SELECT** *

4 **FROM** SOURCE;

100% 13:4

Result Grid Filter Rows: Search Edit: Export/Import:

	source_id	source_name	publish_year	url
1		Reactive Oxygen Species in TNF α -Induced Sig...	2010	https://www.sciencedirect.com/science/article/pii...
2		Physiology, Renin Angiotensin System	2026	https://www.ncbi.nlm.nih.gov/books/NBK470410/
3		Regulation of Rho-Dependent Transcription Ter...	2000	https://pubs.acs.org/doi/10.1021/bi992658z
5		Light-Regulated Nuclear Import and Degradatio...	2010	https://pmc.ncbi.nlm.nih.gov/articles/PMC30370...
6		UniProt	2024	https://www.uniprot.org/uniprotkb/Q9S7Z8/entry
9		Source	2000	https://example.com
	NULL	NULL	NULL	NULL

GENE_ANNOTATION after procedure is called:

Term Project Report: Integrated Gene Knowledge Database

```
1  call add_annotation_with_source('GO:0007267', 1, 'IMP', 'Source', 2000, 'https://example.com');
2
3  • SELECT *
4  FROM SOURCE;
5
6  • SELECT *
7  FROM GENE_ANNOTATION;
```

100% 22:7

Result Grid Filter Rows: Search Edit: Export/Import:

	annotation_id	gene_product_id	go_id	evidence_code	date_created	date_last_update	comment
1	1	1	GO:0006915	IDA	2026-04-09 16:38:52	2026-04-09 16:38:52	TNF induces apoptosis via TNFR1 signaling
2	2	2	GO:0007165	IDA	2026-04-09 16:38:52	2026-04-09 16:38:52	ACE catalyzes angiotensin I to II, which activates signa...
3	3	3	GO:0009987	IDA	2026-04-09 16:38:52	2026-04-09 16:38:52	Rho protein mediates the termination of transcription in...
5	8	8	GO:0005634	EXP	2026-04-09 16:38:52	2026-04-09 16:56:58	FLC functions as a nuclear transcription factor that repr...
6	9	9	GO:0005737	IEA	2026-04-09 16:38:52	2026-04-09 16:38:52	PHYA is localized in the cytoplasm prior to being activa...
10	1	1	GO:0007267	IMP	2026-04-09 21:24:55	2026-04-09 21:24:55	NULL
	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Looking at the last row of each table, we see that a new record with the values we provided have been created. Thus, the procedure has shown to run successfully. To demonstrate the error output, we provide a `go_id` that doesn't exist. The result (error message) is shown below:

```
1 • call add_annotation_with_source('GO:0000000', 1, 'IMP', 'Source', 2000, 'https://example.com')
```

100% 95:1

Action Output

	Time	Action	Response
1	21:44:05	call add_annotation_with_source('GO:0000000', 1, 'IMP', 'Source', 2000, 'http...	Error Code: 1644. Gene ontology term does not exist

Stored Procedure 2: add_additional_source

This stored procedure allows a user to enter an additional source for a gene annotation and takes four inputs: `annotation_id1` with datatype INT, `source_name1` with datatype VARCHAR(150), `publish_year1` with datatype YEAR, and `url1` with datatype VARCHAR(2048). Shown below is the procedure:

Term Project Report: Integrated Gene Knowledge Database

```
1 • CREATE PROCEDURE `add_additional_source` (  
2     /* Name of inputs */  
3     IN annotation_id1 INT,  
4     IN source_name1 VARCHAR(150),  
5     IN publish_year1 YEAR,  
6     IN url1 VARCHAR(2048)  
7 )  
8 BEGIN  
9     DECLARE use_source_id INT;  
10    /* First, we check to see if the gene annotation exists. If it does, we know it already has one source. */  
11    IF NOT EXISTS (  
12        SELECT 1  
13        FROM GENE_ANNOTATION ga  
14        WHERE ga.annotation_id = annotation_id1  
15    ) THEN  
16        SIGNAL SQLSTATE '45000'  
17        SET MESSAGE_TEXT = 'Gene annotation does not exist';  
18    END IF;  
19    /* Now, we insert the additional source */  
20    INSERT INTO SOURCE (source_name, publish_year, url)  
21    VALUES (source_name1, publish_year1, url1);  
22    SET use_source_id = LAST_INSERT_ID();  
23    /* We insert the annotation_id and source_id into the associative table linking the two entities */  
24    INSERT INTO ANNOTATION_SOURCE (source_id, annotation_id)  
25    VALUES (use_source_id, annotation_id1);  
26 END
```

Now we will show a demonstration of the procedure going into effect. We will add a source to the gene annotation we created in the previous example with valid input values. This is only for demonstration purposes; the values aren't accurate and the inserted record will be later removed from the database. It is important to note that gene annotation must already exist otherwise an error message is displayed. The SOURCE table before and after the procedure is called is shown below:

SOURCE before procedure is called:

```
3 • SELECT *  
4 FROM SOURCE;
```

100% 13:4

Result Grid Filter Rows: Search Edit: Export/Import:

	source_id	source_name	publish_year	url
1	1	Reactive Oxygen Species in TNFα-Induced Sig...	2010	https://www.sciencedirect.com/science/article/pii...
2	2	Physiology, Renin Angiotensin System	2026	https://www.ncbi.nlm.nih.gov/books/NBK470410/
3	3	Regulation of Rho-Dependent Transcription Ter...	2000	https://pubs.acs.org/doi/10.1021/bi992658z
5	5	Light-Regulated Nuclear Import and Degradatio...	2010	https://pmc.ncbi.nlm.nih.gov/articles/PMC30370...
6	6	UniProt	2024	https://www.uniprot.org/uniprotkb/Q9S7Z8/entry
9	9	Source	2000	https://example.com
	NULL	NULL	NULL	NULL

The last row shows the most recent source we added.

SOURCE after procedure is called:


```

1 • call add_additional_source(10, 'Source2', 2000, 'https://example.com');
2
3 • SELECT * FROM SOURCE;

```

100% 22:3

Result Grid Filter Rows: Search Edit: Export/Import:

	source_id	source_name	publish_year	url
1		Reactive Oxygen Species in TNFa-Induced Sig...	2010	https://www.sciencedirect.com/science/article/pii...
2		Physiology, Renin Angiotensin System	2026	https://www.ncbi.nlm.nih.gov/books/NBK470410/
3		Regulation of Rho-Dependent Transcription Ter...	2000	https://pubs.acs.org/doi/10.1021/bi992658z
5		Light-Regulated Nuclear Import and Degradatio...	2010	https://pmc.ncbi.nlm.nih.gov/articles/PMC30370...
6		UniProt	2024	https://www.uniprot.org/uniprotkb/Q9S7Z8/entry
9		Source	2000	https://example.com
10		Source2	2000	https://example.com
	NULL	NULL	NULL	NULL

We can see that a new source has been successfully added. We can also verify that the associative table ANNOTATION_SOURCE has been updated to reflect this change (screenshot shown below):

```

3 • SELECT *
4 FROM ANNOTATION_SOURCE ans
5 WHERE ans.source_id = 10;

```

100% 27:4

Result Grid Filter Rows: Search

	source_id	annotation_id
	10	10
	NULL	NULL

Similar to the last stored procedure, if an invalid annotation_id was referenced, an error message will be displayed. An example is shown below:

```

1 • call add_additional_source(1000, 'Source2', 2000, 'https://example.com');
2

```

100% 32:1

Action Output

	Time	Action	Response
1	21:55:54	call add_additional_source(1000, 'Source2', 2000, 'https://example.com')	Error Code: 1644. Gene annotation does not exist

Stored Procedure 3: add_go_term

This stored procedure allows a user to enter a gene ontology term and takes four inputs: go_id1 with datatype VARCHAR(15), term_name1 with datatype VARCHAR(200), ontology1 with datatype VARCHAR(2), and definition1 with datatype VARCHAR(500). Shown below is the procedure:

```

1 • CREATE PROCEDURE `add_go_term` (
2     /* Name of inputs */
3     IN go_id1 VARCHAR(15),
4     IN term_name1 VARCHAR(200),
5     IN ontology1 VARCHAR(2),
6     IN definition1 VARCHAR(500)
7 )
8 BEGIN
9     /* First, we check to see if the go_id already exists */
10    IF EXISTS (
11        SELECT 1
12        FROM GO_TERM gt
13        WHERE gt.go_id = go_id1
14    ) THEN
15        SIGNAL SQLSTATE '45000'
16        SET MESSAGE_TEXT = 'Gene ontology term already exists';
17    END IF;
18    /* Now, we insert the go term */
19    INSERT INTO GO_TERM (go_id, term_name, ontology, definition, date_created, date_last_update)
20    VALUES (go_id1, term_name1, ontology1, definition1, NOW(), NOW());
21 END

```

Now we will show a demonstration of the procedure going into effect. We will add a go term with valid input values. This is only for demonstration purposes; the values aren't accurate and the inserted record will be later removed from the database. It is important to note that the go term must be unique and can't already exist in the database. The GO_TERM table before and after the procedure is called is shown below.

GO_TERM before procedure is called:

1 •	SELECT *
2	FROM GO_TERM;

100%	14:2	Result Grid	Filter Rows: Search	Edit: [Icons]	Export/Import: [Icons]
go_id	term_name	ontology	definition	date_created	date_last_update
GO:0005634	nucleus	cc	A membrane bound organelle in eukaryotic...	2026-04-09 16:38:52	2026-04-09 16:38:52
GO:0005737	cytoplasm	cc	The fluid-like substance that fills the interior...	2026-04-09 16:38:52	2026-04-09 16:42:38
GO:0006915	apoptotic pro...	bp	The process of programmed cell death that...	2026-04-09 16:38:52	2026-04-09 16:38:52
GO:0007154	cell communi...	bp	Any process by which a cell gives and recei...	2026-04-09 16:38:52	2026-04-09 16:38:52
GO:0007165	signal transd...	bp	The cellular process where a signal is trans...	2026-04-09 16:38:52	2026-04-09 16:38:52
GO:0007267	cell-cell sign...	bp	Any process that mediates the transfer of in...	2026-04-09 16:38:52	2026-04-09 16:38:52
GO:0008283	cell proliferati...	bp	The process by which cells grow and divide...	2026-04-09 16:38:52	2026-04-09 16:38:52
GO:0009987	cellular process	bp	Any process that is carried out at the cellula...	2026-04-09 16:38:52	2026-04-09 16:38:52
GO:0010647	positive regul...	bp	A process that increases the rate of cell co...	2026-04-09 16:38:52	2026-04-09 16:38:52
GO:0012501	programmed...	bp	A highly regulated process of cellular self-d...	2026-04-09 16:38:52	2026-04-09 16:38:52
GO:0035426	extracellular...	bp	Any process that mediates the transfer of in...	2026-04-09 16:38:52	2026-04-09 16:38:52
NULL	NULL	NULL	NULL	NULL	NULL

Term Project Report: Integrated Gene Knowledge Database

GO_TERM after procedure is called:

```
1 • call add_go_term('GO:0000000', 'nonexistent term', 'cc', 'nonexistent definition' );
2
3 • SELECT *
4 FROM GO_TERM;
```

100% 14:4

Result Grid Filter Rows: Search Edit: Export/Import:

go_id	term_name	ontology	definition	date_created	date_last_update
GO:0000000	nonexistent term	cc	nonexistent definition	2026-04-09 22:14:28	2026-04-09 22:14:28
GO:0005634	nucleus	cc	A membrane bound organelle in eukaryotic...	2026-04-09 16:38:52	2026-04-09 16:38:52
GO:0005737	cytoplasm	cc	The fluid-like substance that fills the interior...	2026-04-09 16:38:52	2026-04-09 16:42:38
GO:0006915	apoptotic process	bp	The process of programmed cell death that...	2026-04-09 16:38:52	2026-04-09 16:38:52
GO:0007154	cell communication	bp	Any process by which a cell gives and recei...	2026-04-09 16:38:52	2026-04-09 16:38:52
GO:0007165	signal transduction	bp	The cellular process where a signal is trans...	2026-04-09 16:38:52	2026-04-09 16:38:52
GO:0007267	cell-cell signaling	bp	Any process that mediates the transfer of in...	2026-04-09 16:38:52	2026-04-09 16:38:52
GO:0008283	cell proliferation	bp	The process by which cells grow and divide...	2026-04-09 16:38:52	2026-04-09 16:38:52
GO:0009987	cellular process	bp	Any process that is carried out at the cellula...	2026-04-09 16:38:52	2026-04-09 16:38:52
GO:0010647	positive regulation of cell communication	bp	A process that increases the rate of cell co...	2026-04-09 16:38:52	2026-04-09 16:38:52
GO:0012501	programmed cell death	bp	A highly regulated process of cellular self-d...	2026-04-09 16:38:52	2026-04-09 16:38:52
GO:0035426	extracellular matrix-cell signaling	bp	Any process that mediates the transfer of in...	2026-04-09 16:38:52	2026-04-09 16:38:52

We can see that the procedure ran successfully as a new record was added (record is at the top this time). If we try to call the procedure again with the same values, we get an error since the go_id already exists:

```
1 • call add_go_term('GO:0000000', 'nonexistent term', 'cc', 'nonexistent definition' );
2
```

100% 85:1

Action Output

	Time	Action	Response
1	22:16:36	call add_go_term('GO:0000000', 'nonexistent term', 'cc', 'nonexistent definiti...	Error Code: 1644. Gene ontology term already exists

Stored Procedure 4: add_gene_product_for_gene

This stored procedure allows a user to enter a gene product for a particular gene and takes three inputs: gene_id1 with datatype INT, product_name1 with datatype VARCHAR(100), and product_type1 with datatype VARCHAR(7). Shown below is the procedure:

```

1 • CREATE PROCEDURE `add_gene_product_for_gene` (
2     /* Name of inputs */
3     IN gene_id1 INT,
4     IN product_name1 VARCHAR(100),
5     IN product_type1 VARCHAR(7)
6 )
7 BEGIN
8     /* First, we check to see if the gene_id exists */
9     IF NOT EXISTS (
10         SELECT 1
11         FROM GENE g
12         WHERE g.gene_id = gene_id1
13     ) THEN
14         SIGNAL SQLSTATE '45000'
15         SET MESSAGE_TEXT = 'Gene does not exist';
16     END IF;
17     /* Second, we check to see if the gene_product_id already exists for the given gene */
18     IF EXISTS (
19         SELECT 1
20         FROM GENE_PRODUCT gp
21         WHERE gp.product_name = product_name1
22             AND gp.gene_id = gene_id1
23     ) THEN
24         SIGNAL SQLSTATE '45000'
25         SET MESSAGE_TEXT = 'Gene product already exists';
26     END IF;
27     /* Now, we insert the gene product */
28     INSERT INTO GENE_PRODUCT (product_name, product_type, gene_id)
29     VALUES (product_name1, product_type1, gene_id1);
30 END

```

Now we will show a demonstration of the procedure going into effect. We will add a gene product with valid input values for the gene with `gene_id = 1`. This is only for demonstration purposes; the values aren't accurate and the inserted record will be later removed from the database. It is important to note that the gene product must be unique and can't already exist. The `GENE_PRODUCT` table before and after the procedure is called is shown below.

`GENE_PRODUCT` before procedure is called:

Term Project Report: Integrated Gene Knowledge Database

```

1 • SELECT *
2 FROM GENE_PRODUCT;

```

100% 19:2

Result Grid Filter Rows: Search Edit: Export/Import:

	gene_product_id	gene_id	product_name	product_type	description
1		4	tumor necrosis factor	protein	cytokine
2		10	ACE protein	protein	angiotensin I converting enzyme
3		20	Rho protein	protein	transcription termination factor
4		9	BRCA1 protein	protein	nuclear phosphoprotein
5		24	BRCA2 protein	protein	nuclear DNA repair protein
6		25	PALB2 protein	protein	partner and localizer of BRCA2 protein
7		15	phytochrome B	protein	a plant photoreceptor that forms a photobody
8		16	FLC protein	protein	a MADS-box transcription factor that represses...
9		17	phytochrome A	protein	a photoreceptor responsible for sensing far-red l...
	NULL	NULL	NULL	NULL	NULL

GENE_PRODUCT after procedure is called:

```

1 • call add_gene_product_for_gene(1, 'nonexistent product', 'rna');
2
3 • SELECT *
4 FROM GENE_PRODUCT;

```

100% 19:4

Result Grid Filter Rows: Search Edit: Export/Import:

	gene_product_id	gene_id	product_name	product_type	description
1		4	tumor necrosis factor	protein	cytokine
2		10	ACE protein	protein	angiotensin I converting enzyme
3		20	Rho protein	protein	transcription termination factor
4		9	BRCA1 protein	protein	nuclear phosphoprotein
5		24	BRCA2 protein	protein	nuclear DNA repair protein
6		25	PALB2 protein	protein	partner and localizer of BRCA2 protein
7		15	phytochrome B	protein	a plant photoreceptor that forms a photobody
8		16	FLC protein	protein	a MADS-box transcription factor that represses...
9		17	phytochrome A	protein	a photoreceptor responsible for sensing far-red l...
10		1	nonexistent product	rna	NULL
	NULL	NULL	NULL	NULL	NULL

We can see that a new entry was created at the bottom. This shows that the procedure was successful. If we try to call the same procedure with the same values again, we will encounter an error. This is shown below:

```

1 • call add_gene_product_for_gene(1, 'nonexistent product', 'rna');

```

100% 65:1

Action Output

	Time	Action	Response
✖ 1	22:31:07	call add_gene_product_for_gene(1, 'nonexistent product', 'rna')	Error Code: 1644. Gene product already exists

Stored Procedure 5: add_gene_for_organism

This stored procedure allows a user to enter a gene for a particular organism and takes six inputs: organism_id1 with datatype INT, gene_symbol1 with datatype VARCHAR(20), chromosome1 with datatype VARCHAR(20), start_position1 with datatype INT, end_position1 with datatype INT, and strand1 with datatype CHAR(1). Shown below is the procedure:

```

1  CREATE PROCEDURE `add_gene_for_organism` (
2      /* Name of inputs */
3      IN organism_id1 INT,
4      IN gene_symbol1 VARCHAR(20),
5      IN chromosome1 VARCHAR(20),
6      IN start_position1 INT,
7      IN end_position1 INT,
8      IN strand1 CHAR(1)
9  )
10 BEGIN
11     /* First, we check to see if the organism exists */
12     IF NOT EXISTS (
13         SELECT 1
14         FROM ORGANISM o
15         WHERE o.organism_id = organism_id1
16     ) THEN
17         SIGNAL SQLSTATE '45000'
18         SET MESSAGE_TEXT = 'Organism does not exist';
19     END IF;
20     /* Second, we check to see if the gene already exists */
21     IF EXISTS (
22         SELECT 1
23         FROM GENE g
24         WHERE g.gene_symbol = gene_symbol1
25             AND g.organism_id = organism_id1
26     ) THEN
27         SIGNAL SQLSTATE '45000'
28         SET MESSAGE_TEXT = 'Gene already exists for this organism';
29     END IF;
30     /* Now, we insert the gene */
31     INSERT INTO GENE (gene_symbol, chromosome, start_position, end_position, strand, organism_id)
32     VALUES (gene_symbol1, chromosome1, start_position1, end_position1, strand1, organism_id1);
33 END

```

Now we will show a demonstration of the procedure going into effect. We will add a gene for an organism with valid input values. This is only for demonstration purposes; the values aren't accurate and the inserted record will be later removed from the database. It is important to note that the gene must not already exist and the organism the gene is being inserted for should exist. The GENE table before and after the procedure is called is shown below:

GENE table before procedure is called:

Term Project Report: Integrated Gene Knowledge Database

1	SELECT *
2	FROM GENE;

100%
11:2

Result Grid
Filter Rows:
Edit:
Export/Import:

	gene_id	gene_symbol	short_descripti...	chromosome	start_positi...	end_positi...	strand	organism_id
1	TP53	NULL	17	7668421	7687490	-	1	
2	EGFR	NULL	7	55019017	55211628	+	1	
3	APOE	NULL	19	44905796	44909393	+	1	
4	TNF	NULL	6	31575565	31578336	+	1	
5	IL6	NULL	7	22727200	22731998	+	1	
6	VEGFA	NULL	6	43770211	43786487	+	1	
7	TGFB1	NULL	19	41330323	41353922	-	1	
8	MTHFR	NULL	1	11785723	11805964	-	1	
9	BRCA1	NULL	17	43044295	43170327	-	1	
10	ACE	NULL	17	63477061	63498373	+	1	
11	STAT3	NULL	17	42313324	42388442	-	1	
12	Blm	NULL	3R	11736810	11741783	-	2	
13	N	NULL	X	3134870	3172221	+	2	
14	dpp	NULL	2L	2428372	2459823	+	2	
15	PHYB	NULL	2	8139756	8144461	+	3	
16	FLC	NULL	5	3173382	3179448	-	3	
17	PHYA	NULL	1	3095160	3100819	-	3	
18	recA	NULL	1	2822708	2823769	-	4	
19	dnaA	NULL	1	3882326	3883729	-	4	
20	rho	NULL	1	3966417	3967676	+	4	
21	daf-16	NULL	1	10750498	10776703	+	5	
22	daf-2	NULL	3	2994514	3040846	-	5	
23	skn-1	NULL	4	5651039	5660384	-	5	
24	BRCA2	NULL	13	32315077	32400268	+	1	
25	PALB2	NULL	16	23603165	23641310	-	1	
26	TRP53	NULL	11	69471174	69482699	+	6	
27	TRP63	NULL	16	25502513	25710842	+	6	
28	TRP73	NULL	4	154140706	154224332	-	6	
29	TP63	NULL	3	189596746	189897276	+	1	
	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

GENE table after procedure is called:

Term Project Report: Integrated Gene Knowledge Database

```

1 • call add_gene_for_organism(1, 'ABC', '23', 1000, 2000, '+');
2
3 • SELECT *
4 FROM GENE;

```

100% 9:3

Result Grid
Filter Rows: Search
Edit:
Export/Import:

	gene_id	gene_symbol	short_descripti...	chromosome	start_positi...	end_positi...	strand	organism_id
	1	TP53	NULL	17	7668421	7687490	-	1
	2	EGFR	NULL	7	55019017	55211628	+	1
	3	APOE	NULL	19	44905796	44909393	+	1
	4	TNF	NULL	6	31575565	31578336	+	1
	5	IL6	NULL	7	22727200	22731998	+	1
	6	VEGFA	NULL	6	43770211	43786487	+	1
	7	TGFB1	NULL	19	41330323	41353922	-	1
	8	MTHFR	NULL	1	11785723	11805964	-	1
	9	BRCA1	NULL	17	43044295	43170327	-	1
	10	ACE	NULL	17	63477061	63498373	+	1
	11	STAT3	NULL	17	42313324	42388442	-	1
	12	Blm	NULL	3R	11736810	11741783	-	2
	13	N	NULL	X	3134870	3172221	+	2
	14	dpp	NULL	2L	2428372	2459823	+	2
	15	PHYB	NULL	2	8139756	8144461	+	3
	16	FLC	NULL	5	3173382	3179448	-	3
	17	PHYA	NULL	1	3095160	3100819	-	3
	18	recA	NULL	1	2822708	2823769	-	4
	19	dnaA	NULL	1	3882326	3883729	-	4
	20	rho	NULL	1	3966417	3967676	+	4
	21	daf-16	NULL	1	10750498	10776703	+	5
	22	daf-2	NULL	3	2994514	3040846	-	5
	23	skn-1	NULL	4	5651039	5660384	-	5
	24	BRCA2	NULL	13	32315077	32400268	+	1
	25	PALB2	NULL	16	23603165	23641310	-	1
	26	TRP53	NULL	11	69471174	69482699	+	6
	27	TRP63	NULL	16	25502513	25710842	+	6
	28	TRP73	NULL	4	154140706	154224332	-	6
	29	TP63	NULL	3	189596746	189897276	+	1
	30	ABC	NULL	23	1000	2000	+	1
	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

We can see that a new record was added to the bottom (record with gene_id = 30). Thus, we have shown that the procedure has run successfully. If we try to call the same procedure, we will encounter an error. An example is shown below:

```

1 • call add_gene_for_organism(1, 'ABC', '23', 1000, 2000, '+');

```

100% 29:1

Action Output

	Time	Action	Response
1	22:47:46	call add_gene_for_organism(1, 'ABC', '23', 1000, 2000, '+')	Error Code: 1644. Gene already exists for this organism

If we try to insert a gene for an organism that doesn't exist, we will encounter an error. An example for this is also shown (there is no gene in the database corresponding to gene_id = 20):

1	•	call add_gene_for_organism(20, 'ABC', '23', 1000, 2000, '+');
100% 44:1		
Action Output		
	Time	Action
		Response
1	22:46:25	call add_gene_for_organism(20, 'ABC', '23', 1000, 2000, '+') Error Code: 1644. Organism does not exist

4.3 Queries

We will now cover 8 queries that are commonly used in this database. These queries demonstrate how users can retrieve and analyze important information.

4.3.1 Gene Products Associated with a Genetic Disease

Query name: retrieve_gene_products_by_disease

This query will retrieve all gene products that are produced by genes associated with a particular disease. The value stored in the '@disease_name' variable can be modified by the user to specify the disease. In addition to the gene product name, the query displays the disease name and gene symbol for clarity. In the example shown below, 3 gene products associated with breast cancer were retrieved.

```

1 • SET @disease_name = 'breast cancer';
2 • SELECT
3     gd.disease_name,
4     g.gene_symbol,
5     gp.product_name
6 FROM GENE_PRODUCT as gp
7 JOIN GENE as g
8 ON g.gene_id = gp.gene_id
9 JOIN GENE_DISEASE_ASSOCIATION as gda
10 ON gp.gene_id = gda.gene_id
11 JOIN GENE_DISEASE as gd
12 ON gda.disease_id = gd.disease_id
13 WHERE gd.disease_name = @disease_name;

```

100% 39:13

Result Grid

Filter Rows:

Export

	disease_name	gene_symbol	product_name
	breast cancer	BRCA1	BRCA1 protein
	breast cancer	BRCA2	BRCA2 protein
	breast cancer	PALB2	PALB2 protein

4.3.2 Total Genes in a Gene Family for Each Organism

Query name: count_genes_by_family_and_organism

This query will retrieve the number of total genes belonging to a gene family for each organism. Only organisms with at least 1 gene participating in the gene family will be shown. This is to ensure that only meaningful data is shown. The value stored in the '@family_name' variable can be modified by the user to specify the gene family. The query displays the family name, scientific name, and the count. In the example shown below, 2 organisms contained genes belonging to the p53 family. 'Homo sapiens' had 2 genes belonging to the family and 'Mus musculus' had 3 genes belonging to the family.

```

1 • SET @family_name = 'p53 family';
2 • SELECT
3     gf.family_name,
4     o.scientific_name,
5     COUNT(g.gene_id) as count
6 FROM GENE as g
7 JOIN GENE_FAMILY_MEMBER as gfm
8 ON g.gene_id = gfm.gene_id
9 JOIN GENE_FAMILY as gf
10 ON gfm.family_id = gf.family_id
11 JOIN ORGANISM as o
12 ON g.organism_id = o.organism_id
13 WHERE gf.family_name = @family_name
14 GROUP BY gf.family_name, o.scientific_name;

```

100% 44:14

Result Grid Filter Rows: Search Export:

family_name	scientific_name	count
p53 family	Homo sapiens	2
p53 family	Mus musculus	3

4.3.3 Finding Pathways Associated with a Genetic Disease

Query name: retrieve_pathways_by_disease

This query will retrieve the genetic pathways that are associated with genes that are linked to a genetic disease. The value stored in the '@disease_name' variable can be modified by the user to specify the name of the genetic disease. In addition to the name of the pathway, the query also displays the name of the disease for clarity. In the example shown below, there are 4 pathways associated with breast cancer.

```

1 • SET @disease_name = 'breast cancer';
2 • SELECT
3     gp.pathway_name,
4     gd.disease_name
5 FROM GENE as g
6 JOIN GENE_DISEASE_ASSOCIATION gda
7 ON g.gene_id = gda.gene_id
8 JOIN GENE_DISEASE gd
9 ON gda.disease_id = gd.disease_id
10 JOIN GENE_PATHWAY_ASSOCIATION gpa
11 ON g.gene_id = gpa.gene_id
12 JOIN GENE_PATHWAY gp
13 ON gpa.pathway_id = gp.pathway_id
14 WHERE gd.disease_name = @disease_name;
15

```

100% 39:14

Result Grid   Filter Rows: Export:

	pathway_name	disease_name	
☐	cell cycle pathway	breast cancer	
☐	apoptosis pathway	breast cancer	
☐	MAPK signaling pathway	breast cancer	
☐	cell cycle pathway	breast cancer	

4.3.4 Gene Annotations Meeting Gene Product and GO Term Criteria

Query name: retrieve_go_annotations_filtered

This query will retrieve the gene annotations for gene ontology terms that belong to a specific gene ontology category, and for gene products that belong to the particular chromosome of a particular organism. This query is highly specific and is one of the many frequently used queries that users of this database may use. The value stored in the '@scientific_name' variable can be modified by the user to specify the scientific name of the organism. The value stored in the '@chromosome' variable can be modified by the user to specify the name of the chromosome. The value stored in the '@ontology' variable can be modified by the user to specify the ontology category (such as 'cc'). The query displays the attributes found in GENE_ANNOTATION: the annotation_id, the name of the gene product, the name of the gene ontology term, the evidence code, date of creation, date of last update, and any comments. In the example shown below, we retrieved a single gene annotation that satisfies the criteria provided by the user.

```

1 • SET @scientific_name = 'Homo sapiens';
2 • SET @chromosome = '6';
3 • SET @ontology = 'bp';
4 • SELECT
5     ga.annotation_id,
6     gp.product_name,
7     gt.term_name,
8     ga.evidence_code,
9     ga.date_created,
10    ga.date_last_update,
11    ga.comment
12 FROM GENE_ANNOTATION ga
13 JOIN GENE_PRODUCT gp
14 ON ga.gene_product_id = gp.gene_product_id
15 JOIN GENE g
16 ON gp.gene_id = g.gene_id
17 JOIN GO_TERM gt
18 ON ga.go_id = gt.go_id
19 JOIN ORGANISM o
20 ON g.organism_id = o.organism_id
21 WHERE g.chromosome = @chromosome
22     AND gt.ontology = @ontology
23     AND o.scientific_name = @scientific_name
24 ORDER BY gp.product_name;

```

100% 26:24

Result Grid Filter Rows: Search Export:

annotation_id	product_name	term_name	evidence_code	date_created	date_last_update	comment
1	tumor necrosis factor	apoptotic process	IDA	2026-04-09 16:38:52	2026-04-09 16:38:52	TNF induces apoptosis via TNFR1 signaling

4.3.5 Gene Product and GO Term Annotation Status

Query name: retrieve_annotation_status_by_species

This query will retrieve all possible combinations of gene products and gene ontology terms for a given species and identifies whether each pair is annotated or missing an annotation. The value stored in the '@scientific_name' variable can be modified by the user to specify the scientific name of the organism. The query displays the product name, gene ontology term name, status, and the scientific name for clarification. The status tells the user if there is a gene annotation for each combination of gene product and go term (unordered). If there is, there is a value of 'ANNOTATED', otherwise there is a value of 'MISSING'. In the example shown below, we can see that there is a gene annotation for the gene product 'Rho protein' and the go term 'cellular process'. All of the other combinations don't have a gene annotation associated with them. The results of this query should only be interpreted for the organism with the scientific name 'Escherichia coli'.

```

1 • SET @scientific_name = 'Escherichia coli';
2 SELECT
3     gp.product_name,
4     gt.term_name,
5     CASE
6         WHEN ga.annotation_id IS NULL THEN 'MISSING'
7         ELSE 'ANNOTATED'
8     END AS STATUS,
9     (SELECT o.scientific_name
10      FROM GENE g
11      JOIN ORGANISM o
12      ON g.organism_id = o.organism_id
13      WHERE g.gene_id = gp.gene_id
14     ) AS scientific_name
15 FROM GENE_PRODUCT gp
16 CROSS JOIN GO_TERM gt
17 LEFT JOIN GENE_ANNOTATION ga
18     ON gp.gene_product_id = ga.gene_product_id
19     AND gt.go_id = ga.go_id
20 WHERE gp.gene_id IN (
21     SELECT g.gene_id
22     FROM GENE g
23     JOIN ORGANISM o
24     ON g.organism_id = o.organism_id
25     WHERE o.scientific_name = @scientific_name
26 );

```

100% 3:26

Result Grid Filter Rows: Search Export:

product_name	term_name	STATUS	scientific_name
Rho protein	apoptotic process	MISSING	Escherichia coli
Rho protein	cell communication	MISSING	Escherichia coli
Rho protein	cell proliferation	MISSING	Escherichia coli
Rho protein	cell-cell signaling	MISSING	Escherichia coli
Rho protein	cellular process	ANNOTATED	Escherichia coli
Rho protein	cytoplasm	MISSING	Escherichia coli
Rho protein	extracellular matrix...	MISSING	Escherichia coli
Rho protein	nucleus	MISSING	Escherichia coli
Rho protein	positive regulation...	MISSING	Escherichia coli
Rho protein	programmed cell d...	MISSING	Escherichia coli
Rho protein	signal transduction	MISSING	Escherichia coli

4.3.6 Retrieve Child Gene Ontology Terms

Query name: retrieve_child_go_terms

This query will retrieve all of the child gene ontology terms associated with a parent gene ontology term. The value stored in the '@parent_term_name' variable can be modified by the user to specify the parent gene ontology term whose children are being searched for. The query displays the child term and the parent term. In the example shown below, we can see that the gene ontology term 'cell communication' has 4 child gene ontology terms associated with it.

```

1 • SET @parent_term_name = 'cell communication';
2 • WITH RECURSIVE go_descendant AS (
3   SELECT gt.go_id
4   FROM GO_TERM as gt
5   WHERE gt.term_name = @parent_term_name
6
7   UNION
8
9   SELECT gtr.child_go_id
10  FROM GO_TERM_RELATION as gtr
11  JOIN go_descendant as gd
12  ON gtr.parent_go_id = gd.go_id
13 )
14 SELECT
15     gt.term_name as child_term,
16     @parent_term_name as parent_term
17 FROM GO_TERM gt
18 JOIN go_descendant gd
19 ON gt.go_id = gd.go_id
20 WHERE gt.term_name != @parent_term_name;

```

100% 41:20

Result Grid Filter Rows: Search Export:

child_term	parent_term
signal transduction	cell communication
cell-cell signaling	cell communication
extracellular matrix-cell signaling	cell communication
positive regulation of cell communication	cell communication

4.3.7 Retrieve Gene Annotations with Recent Sources

Query name: retrieve_annotations_by_year

This query will retrieve all gene annotations that are associated with sources published on or after a specified year. This is useful for identifying annotations that include at least one relatively recent source. The value stored in the '@publish_year' variable can be modified by the user to specify the oldest acceptable year for the sources associated with the gene annotations they want to search. The query displays the annotation id and the year the source was published. In the example shown below, we can see that there are 4 gene annotations with at least one source published on or after 2010.

1	•	SET	@publish_year = 2010;
2	•	SELECT	
3			ga.annotation_id,
4			s.publish_year
5		FROM	GENE_ANNOTATION ga
6		JOIN	ANNOTATION_SOURCE ans
7		ON	ga.annotation_id = ans.annotation_id
8		JOIN	SOURCE s
9		ON	ans.source_id = s.source_id
10		WHERE	s.publish_year >= @publish_year;

100%	↕	39:10
Result Grid	📄	↕
Filter Rows:	🔍 Search	Export:
annotation_id	publish_year	
1	2010	
2	2026	
5	2010	
6	2024	

4.3.8 Gene Products for Genes Found in Region of Chromosome

Query name: retrieve_gene_products_by_genomic_region

This query will retrieve the gene products of genes found in a location range of a particular chromosome for a particular organism. The value stored in the '@chromosome' variable can be modified by the user to specify the name of the chromosome they want to search. The value stored in the '@region_start' variable and '@region_end' variable specify the genomic range to search. The value stored in the '@scientific_name' variable can be modified by the user to specify the name of the organism they want to search. In addition to the product name and gene symbol, the query shows the scientific name, chromosome, start position, and end position for clarity. In the example shown below, there are 2 gene products associated with genes that overlap the range [40000000, 70000000] on chromosome 17 in Homo sapiens. A gene doesn't need to lie entirely within the range; it is included as long as any portion of it overlaps the specified region.

```

1 • SET @chromosome = '17';
2 • SET @region_start = 70000000;
3 • SET @region_end = 40000000;
4 • SET @scientific_name = 'Homo sapiens';
5 • SELECT
6     gp.product_name,
7     g.gene_symbol,
8     o.scientific_name,
9     g.chromosome,
10    g.start_position,
11    g.end_position
12 FROM GENE_PRODUCT gp
13 JOIN GENE g
14 ON gp.gene_id = g.gene_id
15 JOIN ORGANISM o
16 ON g.organism_id = o.organism_id
17 WHERE g.chromosome = @chromosome
18     AND g.start_position <= @region_start
19     AND g.end_position >= @region_end
20     AND o.scientific_name = @scientific_name;

```

100% 43:20

Result Grid
Filter Rows:
Export:

	product_name	gene_symbol	scientific_name	chromosome	start_position	end_position
	BRCA1 protein	BRCA1	Homo sapiens	17	43044295	43170327
	ACE protein	ACE	Homo sapiens	17	63477061	63498373

5. CRUD Matrix

In this section, we will show the list of entity types and functions used in this database. Then we will present the CRUD Matrix.

5.1 List of Entity Types

The list of entity types is shown below.

- E1: GENE
- E2: ORGANISM
- E3: GENE_DISEASE
- E4: GENE_DISEASE_ASSOCIATION
- E5: GENE_PATHWAY
- E6: GENE_PATHWAY_ASSOCIATION
- E7: GENE_FAMILY
- E8: GENE_FAMILY_MEMBER
- E9: GENE_PRODUCT
- E10: GO_TERM
- E11: GO_TERM_RELATION
- E12: GENE_ANNOTATION
- E13: SOURCE
- E14: ANNOTATION_SOURCE

5.2 List of Functions

The list of functions is shown below.

- F1: retrieve_gene_id
 - a. Read GENE
 - b. Read ORGANISM
- F2: gene_annotation_count
 - a. Read GENE
 - b. Read ORGANISM
 - c. Read GENE_PRODUCT
 - d. Read GENE_ANNOTATION
- F3: gene_organism_count
 - a. Read ORGANISM
 - b. Read GENE
- F4: retrieve_gene_product_id
 - a. Read GENE_PRODUCT
 - b. Read GENE
 - c. Read ORGANISM
- F5: add_annotation_with_source
 - a. Create SOURCE
 - b. Create GENE_ANNOTATION
 - c. Create ANNOTATION_SOURCE
 - d. Read GENE_PRODUCT
 - e. Read GO_TERM
- F6: add_additional_source
 - a. Create SOURCE
 - b. Create ANNOTATION_SOURCE
 - c. Read GENE_ANNOTATION
- F7: add_go_term
 - a. Create GO_TERM
 - b. Read GO_TERM

- F8: add_gene_product_for_gene
 - a. Create GENE_PRODUCT
 - b. Read GENE
 - c. Read GENE_PRODUCT
- F9: add_gene_for_organism
 - a. Create GENE
 - b. Read ORGANISM
 - c. Read GENE
- F10: retrieve_gene_products_by_disease
 - a. Read GENE
 - b. Read GENE_PRODUCT
 - c. Read GENE_DISEASE
 - d. Read GENE_DISEASE_ASSOCIATION
- F11: count_genes_by_family_and_organism
 - a. Read GENE
 - b. Read GENE_FAMILY
 - c. Read GENE_FAMILY_MEMBER
 - d. Read ORGANISM
- F12: retrieve_pathways_by_disease
 - a. Read GENE
 - b. Read GENE_DISEASE
 - c. Read GENE_DISEASE_ASSOCIATION
 - d. Read GENE_PATHWAY
 - e. Read GENE_PATHWAY_ASSOCIATION
- F13: retrieve_go_annotations_filtered
 - a. Read GENE_ANNOTATION
 - b. Read GENE_PRODUCT
 - c. Read GENE
 - d. Read GO_TERM
 - e. Read ORGANISM
- F14: retrieve_annotation_status_by_species
 - a. Read GENE_PRODUCT
 - b. Read GO_TERM
 - c. Read GENE_ANNOTATION
 - d. Read GENE
 - e. Read ORGANISM
- F15: retrieve_child_go_terms
 - a. Read GO_TERM
 - b. Read GO_TERM_RELATION
- F16: retrieve_annotations_by_year
 - a. Read GENE_ANNOTATION
 - b. Read ANNOTATION_SOURCE
 - c. Read SOURCE
- F17: retrieve_gene_products_by_genomic_region
 - a. Read GENE_PRODUCT
 - b. Read GENE
 - c. Read ORGANISM

Term Project Report: Integrated Gene Knowledge Database

The CRUD matrix is shown below.

	E1	E2	E3	E4	E5	E6	E7	E8	E9	E10	E11	E12	E13	E14
F1	R	R												
F2	R	R							R			R		
F3	R	R												
F4	R	R							R					
F5									R	R		C	C	C
F6												R	C	C
F7										RC				
F8	R								RC					
F9	RC	R												
F10	R		R	R					R					
F11	R	R					R	R						
F12	R		R	R	R	R								
F13	R	R							R	R		R		
F14	R	R							R	R		R		
F15										R	R			
F16												R	R	R
F17	R	R							R					

6. Concluding Remarks

This database project has taught me a lot about database design and implementation, how to write queries using SQL, and has provided me with hands-on experience using a variety of functional and application tools. In the design phase, I have learned how to normalize a database to the Third Normal Form and create an ER diagram that accurately represents entities, relationships, and constraints. During the implementation phase, I learned how to translate the conceptual and relational models into a fully developed database. This included using DDL and DML to create tables, define constraints, and enforce integrity rules within MySQL. I was able to develop advanced features such as triggers, functions, and stored procedures to encapsulate database logic and automate complex tasks.

Some of my biggest strengths during this project were my ability to translate requirements into a structured schema, my understanding of integrity constraints, and my ability to debug and ensure that queries and other database components were consistent and accurate. Since this project is related to bioinformatics, a field I am quite familiar with, I had a strong understanding of the underlying business rules and constraints. While I found writing standard queries to be fairly straightforward, more complex queries such as those involving nested subqueries, complex joins, and recursive logic required careful troubleshooting and debugging.

One of my biggest weaknesses was trying to accomplish more than necessary. Initially, I included 19 entities in an effort to make my project look more comprehensive, but I realized that managing the extra entities and relationships was very time-consuming and did not significantly improve the overall design. A more focused design with fewer, well-defined entities would have been equally effective.

If I had more time, I would develop an application layer that would interact with the database. This would make it easier for users of the database to perform operations and manage data without directly writing any SQL. I would also implement indexing to improve query performance. While this database hasn't been populated with millions of entries like those found in modern bioinformatics systems, incorporating indexes would deepen my understanding of performance optimization and how databases are structured in real-world applications.

Overall, this project has significantly improved my understanding of database management systems and has allowed me to apply the concepts I have learned in this course.

Appendices

There are no known defects that have been identified in the system. All of the tested features function as expected.

Appendix A - DDL, INSERT, SELECT Statements

The DDL statements for creating and modifying database objects, INSERT statements to populate test data into the database, and SELECT statements to display the test data are in the pdf file “Appendix A – DDL, INSERT, SELECT Statements” and has been attached to the submission. This file also shows the test data output using the SELECT statements.

Appendix B - Data Dictionary Index

The pdf file “Appendix B – Data Dictionary Index” contains an index to the data dictionary in the format (column_name, table_name) and has been attached to the submission.

References

- Elmasri, R., & Navathe, S. B. (2016). Fundamentals of database systems (7th ed.). Pearson.